WSO2 OpenChoreo

Last Updated: April 2026

Módulo I: Arquitetura Central e Modelo Mental do OpenChoreo

Last Updated: April 2026

Finalidade do módulo

Este módulo estabelece a base conceitual para todo o restante do treinamento.

O OpenChoreo não deve ser ensinado como:

- um portal para subir aplicações;
- uma interface bonita sobre Kubernetes;
- uma coleção solta de ferramentas.

O OpenChoreo deve ser entendido como uma **Internal Developer Platform** modular, Kubernetes-native, multi-plane e API-first, desenhada para transformar intenção declarada em execução operacional governada.

O objetivo deste módulo é formar o modelo mental correto desde o início.

O que é o OpenChoreo no nível arquitetural

O **OpenChoreo** é uma plataforma declarativa e extensível para engenharia de plataforma e desenvolvimento de aplicações em Kubernetes.

Ele reúne:

- APIs orientadas a abstrações;
- Controladores Kubernetes;
- Portal de desenvolvedor baseado em Backstage;
- Automação de workflows;
- Observabilidade integrada;
- Extensibilidade por módulos.

A forma correta de entendê-lo não é como uma lista de funcionalidades, mas como uma plataforma que converte intenção em operação, por meio de APIs, CRDs, controladores e reconciliação contínua.

Mensagem-mestra do OpenChoreo

No OpenChoreo, o usuário declara intenção.

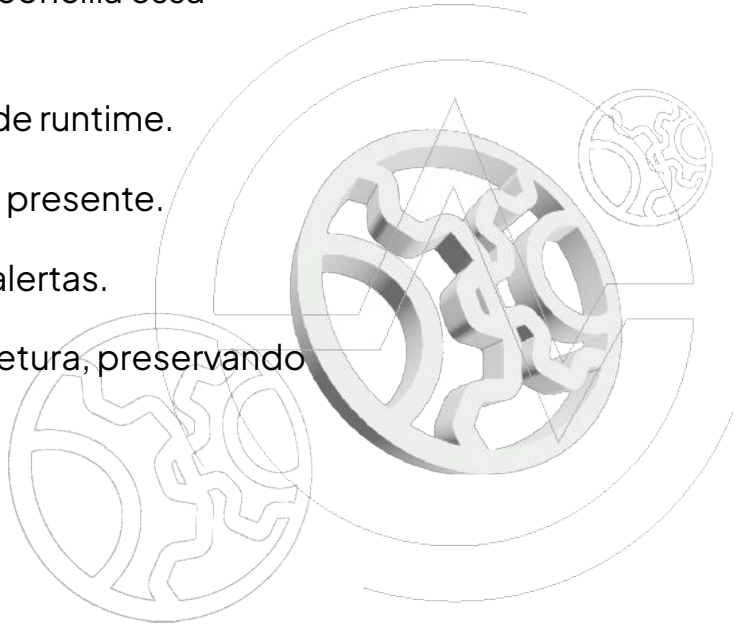
O **Control Plane** interpreta, autentica, autoriza, valida e reconcilia essa intenção.

O **Data Plane** executa os workloads e aplica a semântica de runtime.

O **Workflow Plane** executa builds e automações, quando presente.

O **Observability Plane** centraliza logs, métricas, traces e alertas.

Os módulos conectam as ferramentas concretas à arquitetura, preservando uma experiência unificada de plataforma.



Arquitetura multi-plane

A arquitetura multi-plane é o princípio organizador do OpenChoreo.

Cada plano existe para tratar um domínio operacional distinto:

- Controle;
- Execução de workloads;
- Automação;
- Observabilidade.

Essa separação permite:

- Ciclos de vida independentes;
- Escalabilidade diferenciada;
- Fronteiras próprias de segurança;
- Topologias flexíveis de implantação.

Por isso, a arquitetura multi-plane não é um detalhe de implementação. Ela é parte central do desenho da plataforma.

Control Plane: o cérebro da plataforma

O **Control Plane** é o cérebro do OpenChoreo.

Ele hospeda:

- A API do OpenChoreo;
- As CRDs da plataforma;
- Os controladores;
- As políticas e a lógica de reconciliação.

Seu papel é garantir que o estado desejado, declarado nas APIs de plataforma e de desenvolvedor, seja refletido no estado real dos demais planos.

Seus componentes centrais incluem:

- API Server;
- Controller Manager;
- Cluster Gateway.

O **Control Plane** não é o local onde as aplicações de negócio rodam. Ele é o domínio onde a plataforma pensa, valida, coordena e governa.

Cluster Gateway e o modelo de comunicação

Os demais planos se conectam ao **Control Plane** por conexões de saída.

O **Cluster Gateway** funciona como o hub desse modelo hub-and-spoke.

Ele expõe uma API Secure WebSocket, com conexões de longa duração autenticadas com mTLS.

Esse desenho reduz a necessidade de expor os servidores de API Kubernetes dos planos remotos à internet.

Em termos didáticos, isso mostra que o **OpenChoreo** foi desenhado para coordenação segura entre planos, e não para administração manual de clusters isolados.

Data Plane: onde o runtime acontece

O **Data Plane** é o domínio de runtime do OpenChoreo.

É nele que os workloads efetivamente rodam.

Mas ele não é apenas o lugar onde um pod sobe.

No **Data Plane**, a plataforma aplica:

- Isolamento de projetos e ambientes;
- Topologia de gateway;
- Exposição de endpoints;
- Políticas de rede;
- Segurança de runtime;
- Integração com observabilidade.

O **Data Plane** é, portanto, o local onde a intenção declarada se materializa como sistema em execução governado pela plataforma.

Workflow Plane: builds e automações

O **Workflow Plane** é o domínio de execução de workflows da plataforma.

Ele executa:

- Builds;
- CI workflows;
- Automações de propósito geral;
- Workflows definidos pelo time de plataforma.

O CI nativo documentado do **OpenChoreo** usa Argo Workflows.

Ao mesmo tempo, o **Workflow Plane** é opcional em topologia:

- A instituição pode usar CI externo;
- O **OpenChoreo** continua governando o modelo de plataforma;
- O workload pode ser criado posteriormente pelo pipeline externo.

A ideia central é: o **Workflow Plane** não é apenas “CI”. Ele representa o domínio de automação governada da plataforma.

Observability Plane: telemetria como plano arquitetural

O **Observability Plane** é o plano especializado de observabilidade do **OpenChoreo**.

Ele centraliza:

- Logs;
- Métricas;
- Traces;
- Alertas.

Todos esses dados ficam acessíveis por meio do **Observer API**, que oferece uma interface unificada de consulta.

O **Observability Plane** não deve ser entendido apenas como dashboard ou ferramenta de monitoramento.

Ele é um plano arquitetural próprio, com contratos, módulos e integração nativa com a experiência da plataforma.

Embora seja opcional em topologia, ele é central para diagnóstico operacional e experiência de troubleshooting.

Experience Plane: como o usuário interage

O **Experience Plane** é a camada de interação com a plataforma.

Ele inclui:

- APIs OpenAPI v3;
- CLI occ;
- portal baseado em Backstage;
- MCP servers.

É muito importante não confundir essa camada com os planos operacionais.

O **Experience Plane** não substitui:

- o Control Plane;
- o Data Plane;
- o Workflow Plane;
- o Observability Plane.

Ele é a superfície unificada pela qual usuários, equipes e agentes interagem com a plataforma.

Núcleo da plataforma e planos opcionais

No OpenChoreo:

- Control Plane + Data Plane formam o núcleo funcional da plataforma;
- Workflow Plane + Observability Plane são opcionais em termos de topologia;
- Mesmo assim, os planos opcionais são altamente recomendados para uma experiência completa de IDP.

“Opcional” não significa irrelevante.

Significa que a instituição pode:

- Integrar CI externo;
- Integrar observabilidade externa;
- Manter o modelo arquitetural correto da plataforma.

É necessário compreender o modelo completo, mesmo quando parte da execução concreta ocorre fora do stack padrão.

Extensibilidade por módulos

O **OpenChoreo** foi desenhado como uma IDP modular.

Os módulos do **OpenChoreo** são integrações plugáveis que estendem capacidades da plataforma em pontos de extensibilidade definidos.

Esses módulos cobrem áreas como:

- API Gateway;
- CI;
- observabilidade;
- GitOps.

Um módulo do **OpenChoreo** é um Helm chart que:

- Instala a ferramenta subjacente no cluster-alvo;
- Conecta aos contratos de integração do OpenChoreo;
- Permite que o restante da plataforma interaja com ela corretamente.

Isso significa que o **OpenChoreo** não impõe uma stack única e rígida.

No que capacitar

Em contexto de capacitação, o **OpenChoreo** deve ser ensinado como modelo de plataforma, e não como sequência de telas.

A leitura pedagógica correta é:

- O Control Plane centraliza políticas, abstrações e governança;
- O Data Plane oferece runtime governado para workloads;
- O Workflow Plane padroniza automações e builds, quando adotado;
- O Observability Plane permite diagnóstico estruturado;
- Os módulos permitem adaptação aos padrões institucionais.

Isso permite começar com laboratórios simples e evoluir para ambientes mais maduros sem mudar o modelo mental ensinado aos estudantes.

Erros conceituais a evitar

“OpenChoreo é só um Backstage com plugins.”
Não. O Backstage faz parte da Experience Plane, mas não resume a arquitetura.

“O Data Plane é o próprio OpenChoreo.”
Não. O Data Plane é o domínio de runtime; a coordenação acontece no Control Plane.

“Workflow Plane é apenas CI.”
Não. Ele cobre builds e workflows de automação em sentido mais amplo.

“Observabilidade é um add-on secundário.”
Não. Ela é um plano arquitetural próprio, mesmo quando opcional em topologia.

“Módulos são plugins cosméticos.”
Não. Eles são o mecanismo formal de extensibilidade da plataforma.

Síntese

O **OpenChoreo** é uma **Internal Developer Platform** modular, Kubernetes-native, multi-plane e API-first.

Seu núcleo arquitetural separa claramente os domínios de:

- Controle;
- Runtime;
- Automação;
- Observabilidade.

O usuário declara intenção.

- O **Control Plane** reconcilia essa intenção.
- O **Data Plane** executa os workloads.
- O **Workflow Plane** executa builds e automações, quando presente.
- O **Observability Plane** centraliza a telemetria.
- Os módulos conectam as ferramentas concretas à plataforma sem quebrar a experiência unificada.

Módulo II: Abstrações de Plataforma

Last Updated: April 2026

Conceitos do OpenChoreo: abstrações de plataforma

Este módulo apresenta as abstrações de plataforma do **OpenChoreo**.

Essas abstrações estruturam a topologia da plataforma, definem padrões organizacionais reutilizáveis, controlam a progressão entre ambientes e sustentam o modelo de autosserviço com governança.

No **OpenChoreo**, a plataforma não entrega liberdade irrestrita de configuração.

Ela entrega autosserviço dentro de padrões definidos institucionalmente.

Em contexto universitário, este é o módulo que separa com clareza:

- O que pertence ao estudante;
- O que pertence ao professor;
- O que pertence ao time de plataforma;
- E o que pertence à operação institucional.

Mensagem central do módulo

As abstrações de plataforma do **OpenChoreo** são o mecanismo pelo qual o time de plataforma transforma padrões institucionais, políticas operacionais e topologia de infraestrutura em capacidades reutilizáveis e governadas.

É nelas que se define:

- O que é compartilhado entre todos os times;
- O que é isolado por namespace;
- Onde uma aplicação pode rodar;
- Como ela progride entre ambientes;
- E quais padrões de deploy e automação os desenvolvedores podem consumir.

No **OpenChoreo**, os estudantes consomem abstrações aprovadas.
A plataforma define os guardrails.

O que pertence ao desenvolvedor e o que pertence à plataforma

No **OpenChoreo**, existe uma separação explícita de responsabilidades.

O desenvolvedor trabalha com abstrações de aplicação, como:

- **Project**;
- **Component**;
- **Workload**.

O operador da plataforma trabalha com abstrações de governança e infraestrutura, como:

- Namespace;
- Environment;
- DataPlane;
- WorkflowPlane;
- DeploymentPipeline;
- ComponentType;
- Trait;
- Workflow.

Essa separação é essencial para oferecer autosserviço com segurança, padronização e reprodutibilidade.

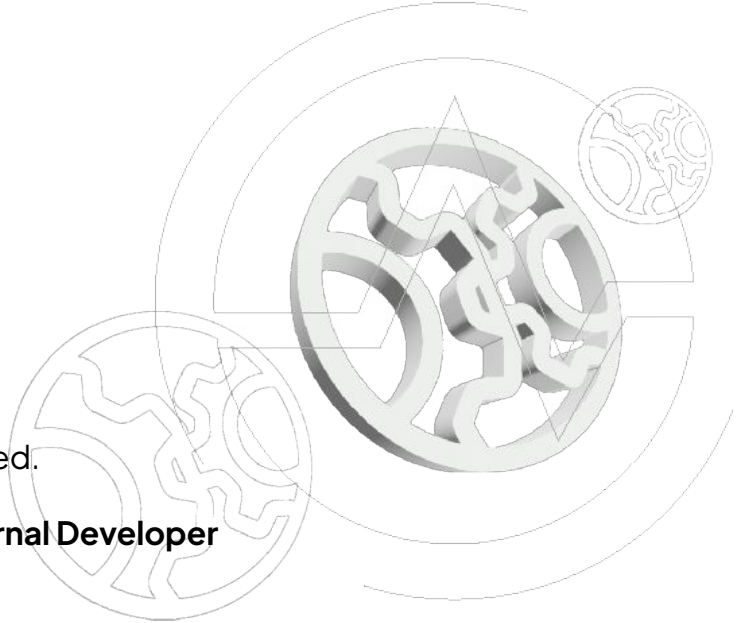
Visão geral das abstrações de plataforma

As abstrações de plataforma do **OpenChoreo** organizam a plataforma em torno de governança, topologia e reutilização.

Neste módulo, o foco recai sobre:

- Namespaces;
- Environments;
- DataPlanes;
- WorkflowPlanes;
- DeploymentPipelines;
- ComponentTypes;
- Traits;
- Workflows;
- E a distinção entre cluster-scoped e namespace-scoped.

Esses recursos formam a base que permite construir uma **Internal Developer Platform** com autosserviço governado.



Namespace: fronteira administrativa do Control Plane

O **OpenChoreo** usa **namespaces** do Kubernetes para organizar e isolar grupos de recursos relacionados.

No **OpenChoreo**, o **namespace** não é apenas um agrupador arbitrário de objetos Kubernetes.

Ele funciona como fronteira administrativa do **Control Plane**.

Por padrão, muitos recursos de plataforma são compartilhados globalmente como **cluster-scoped**.

Já recursos como **Projects**, **Environments** e **DeploymentPipelines** permanecem **namespace-scoped**, porque pertencem ao contexto daquele namespace.

A formulação correta é: **namespace**, no OpenChoreo, é uma fronteira de organização e isolamento da plataforma.

Como o Control Plane identifica namespaces gerenciados

O **OpenChoreo** identifica e gerencia namespaces por meio do rótulo:

openchoreo.dev/control-plane: true

O **Control Plane** usa esse rótulo para:

- Descobrir namespaces;
- Realizar operações de listagem e consulta;
- E organizar recursos de plataforma.

Quando um cluster **OpenChoreo** é criado, o namespace padrão já recebe esse rótulo automaticamente.

Esse detalhe é importante porque mostra que namespace, no **OpenChoreo**, é parte explícita do modelo de governança da plataforma.

O que um novo namespace precisa para operar

Quando um novo **namespace** é criado no **OpenChoreo**, os recursos cluster-scoped padrão já ficam visíveis automaticamente.

Isso inclui, em geral:

- ClusterComponentType;
- ClusterTrait;
- ClusterWorkflow;
- ClusterDataPlane;
- ClusterWorkflowPlane.

Para começar a operar naquele namespace, normalmente basta criar:

- Environments;
- um DeploymentPipeline;
- e um Project.

Essa é uma das bases do autosserviço com governança: padrões globais são compartilhados; contexto local é definido no namespace.

Environment: estágio de entrega e contexto de runtime

Environment é uma abstração de primeira classe no OpenChoreo.

Ele representa um estágio do ciclo de entrega, como:

- Development;
- Staging;
- Production.

Mas **Environment** não é apenas um nome simbólico.

Ele define o contexto em que **workloads** serão implantados e executados.

Cada **Environment** funciona como alvo formal de deploy e promoção dentro da plataforma.

A ideia correta é: **Environment** liga estágio do ciclo de vida a infraestrutura e política operacional.

Environment não é apenas uma label

No **OpenChoreo**, **Environment** não é:

- Apenas uma label;
- Apenas um nome;
- Nem apenas um namespace.

Environment é um recurso formal que aponta para a infraestrutura de destino por meio do **DataPlane** e serve como referência para deploys e promotion workflows.

Ambientes diferentes podem apontar para infraestruturas diferentes.

Por exemplo:

- Development pode usar infraestrutura compartilhada;
- Production pode usar infraestrutura dedicada.

Isso dá ao **OpenChoreo** uma semântica de ambiente muito mais forte do que simples nomenclatura.

DataPlane: infraestrutura de runtime

DataPlane é a abstração que representa um cluster Kubernetes onde workloads de aplicação são implantados.

Ele encapsula a configuração necessária para conectar e gerenciar esse cluster, incluindo:

- Credenciais de conexão;
- Certificados TLS;
- E configurações específicas do cluster.

DataPlane não deve ser considerado como simples cadastro de cluster. Ele é uma abstração operacional de infraestrutura, registrada e governada pelo **OpenChoreo**.

É nele que os workloads de aplicação realmente executam.

WorkflowPlane: infraestrutura de build e automação

WorkflowPlane é a abstração de infraestrutura dedicada à execução de builds e workloads de automação.

Sua função é separar processos intensivos de build e CI do runtime das aplicações.

A documentação atual descreve esse plano como integrado ao Argo Workflows para execução de builds de forma Kubernetes-native.

Se o **WorkflowPlane** não estiver disponível, a plataforma continua apta a implantar imagens pré-construídas, mas não fornece build gerenciado internamente.

A formulação correta é: **DataPlane** executa aplicações; **WorkflowPlane** executa builds e automações.

Comunicação dos planos com o Control Plane

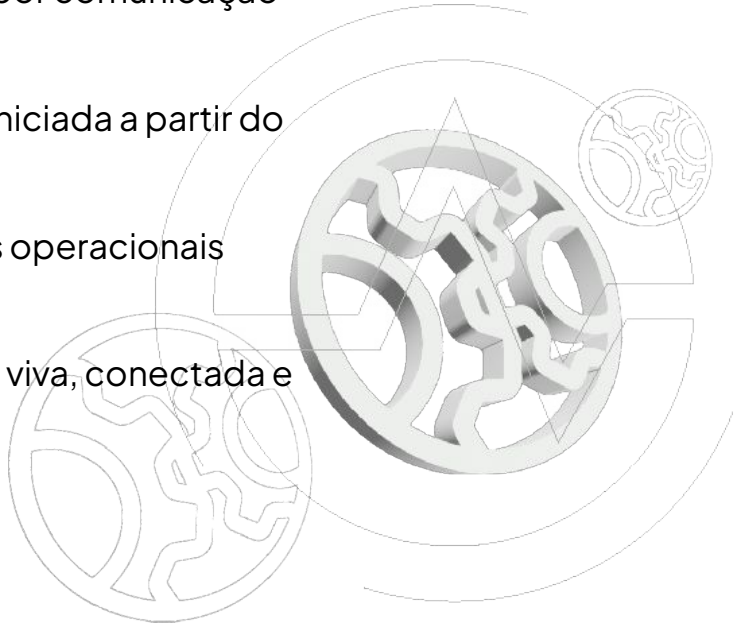
DataPlanes e **WorkflowPlanes** não são registros passivos.

Eles operam como planos conectados ao **Control Plane** por comunicação baseada em agente.

Na arquitetura atual, esses planos usam conexão segura iniciada a partir do próprio plano remoto em direção ao **Control Plane**.

Isso reforça que essas abstrações representam domínios operacionais ativos, e não apenas objetos de inventário.

Em termos didáticos: a plataforma governa infraestrutura viva, conectada e reconciliada continuamente.



ComponentType: padrão institucional de deploy

ComponentType é um template definido por platform engineers que determina como componentes são implantados e gerenciados no **OpenChoreo**.

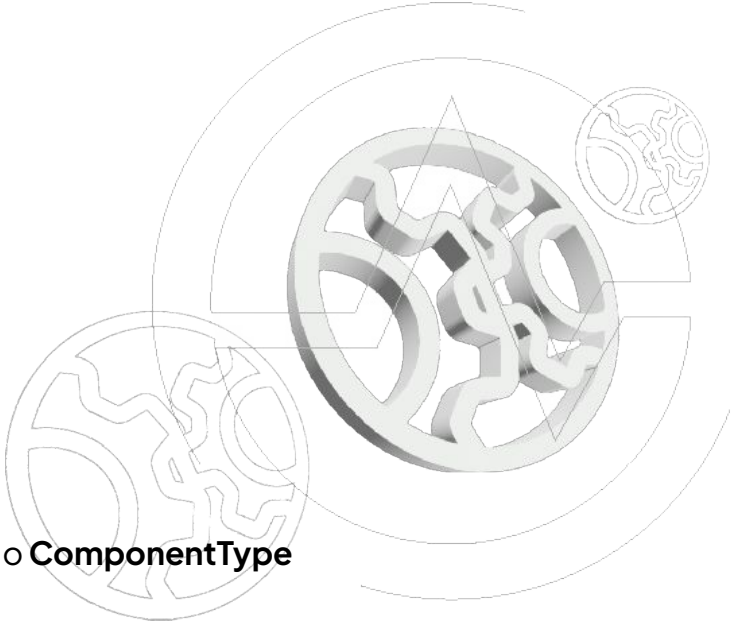
Ele é a ponte entre:

- A intenção do desenvolvedor;
- E a governança da plataforma.

Um **ComponentType** define:

- O tipo principal de workload;
- O schema de parâmetros;
- Os templates de recursos;
- Os workflows permitidos;
- E os traits permitidos ou embutidos.

O platform engineer cria um **Component**. A plataforma define o **ComponentType** que governa esse **Component**.



O que o ComponentType permite controlar

No **OpenChoreo**, **ComponentType** é onde vivem os padrões institucionais de deploy.

Ele permite padronizar, por exemplo:

- Serviços web;
- Aplicações web;
- Tarefas agendadas;
- Jobs;
- Proxies.

A documentação atual mostra que os **ComponentTypes** podem definir:

- Workload types;
- Parâmetros configuráveis;
- Environment overrides;
- Resource templates;
- Workflows permitidos;
- E traits permitidos.

Portanto, **ComponentType** não é um modelo visual do portal. Ele é um contrato governado de implantação.

Traits: capacidades transversais reutilizáveis

Trait é a abstração de plataforma usada para adicionar capacidades transversais a componentes.

Traits complementam o padrão base de deploy sem alterar o **ComponentType** principal.

Eles podem ser usados para acrescentar capacidades como:

- Observabilidade;
- Storage persistente;
- Segurança adicional;
- Autenticação;
- Integrações operacionais.

A formulação correta é: **ComponentType** define o padrão base; **Trait** adiciona capacidades ortogonais e composáveis sobre esse padrão.

Como Traits operam na prática

Traits não são detalhes cosméticos.

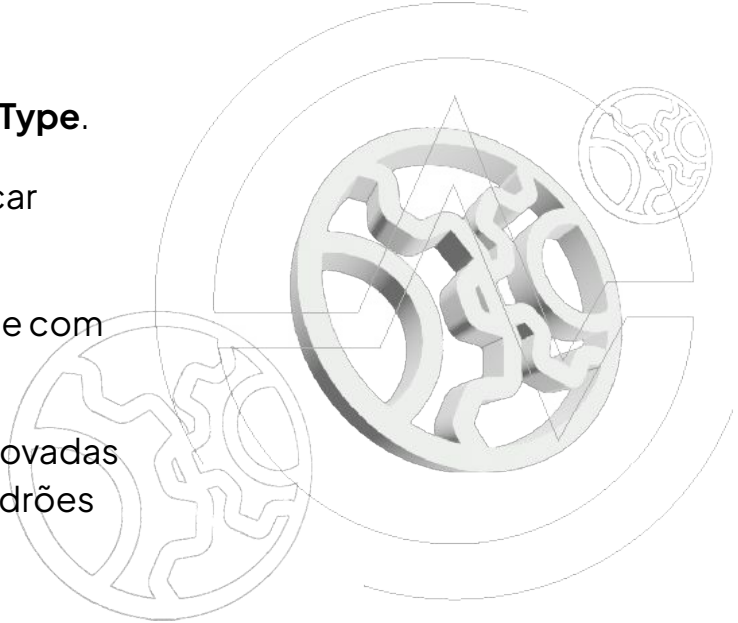
Eles podem:

- Criar novos recursos Kubernetes;
- Ou modificar recursos já gerados pelo **ComponentType**.

Isso permite anexar capacidades adicionais sem multiplicar desnecessariamente o número de **ComponentTypes**.

Em termos pedagógicos, **traits** implementam flexibilidade com governança.

A instituição pode oferecer capacidades reutilizáveis aprovadas institucionalmente, sem deixar cada equipe reinventar padrões operacionais por conta própria.



Workflows e ClusterWorkflows

Workflow é um template de automação definido pela plataforma.

Ele pode ser usado para:

- Build de componentes;
- Automações genéricas;
- Testes;
- Provisionamento;
- Publicação de artefatos;
- Outros processos operacionais.

Workflow não deve ser entendido apenas como pipeline de build.

A variante namespace-scoped é o **Workflow**.

A variante cluster-scoped é o **ClusterWorkflow**, compartilhado entre todos os namespaces.

Governança de workflows por allowedWorkflows

No **OpenChoreo**, o usuário não escolhe livremente qualquer workflow.

A plataforma controla isso por meio do campo:

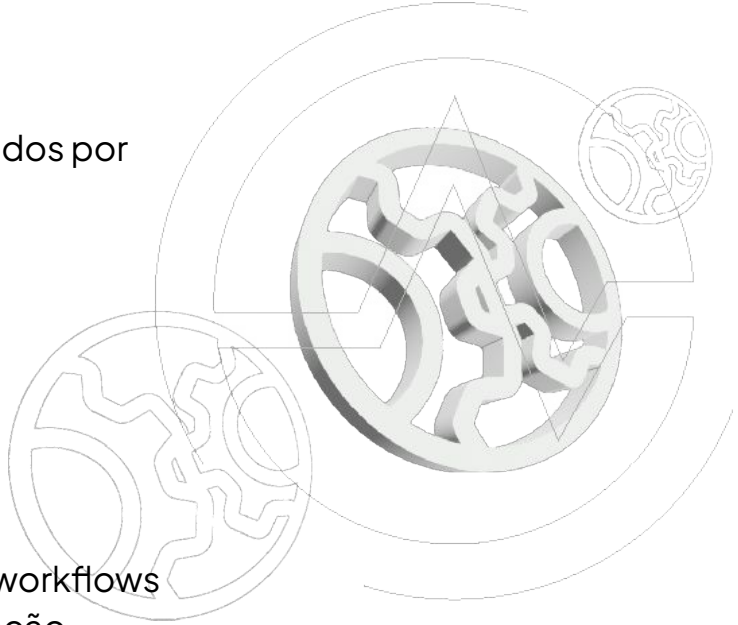
ComponentType.spec.allowedWorkflows

Apenas workflows listados nesse allow list podem ser usados por componentes daquele tipo.

Isso permite impor:

- Padrões de build;
- Ferramentas aprovadas;
- Requisitos de segurança;
- E políticas institucionais por tipo de componente.

A formulação didática correta é: o engenheiro consome workflows autorizados; a plataforma continua governando a automação.



DeploymentPipeline: política declarativa de progressão

DeploymentPipeline define os caminhos permitidos de progressão entre ambientes.

Ele representa o processo organizacional de entrega como configuração declarativa.

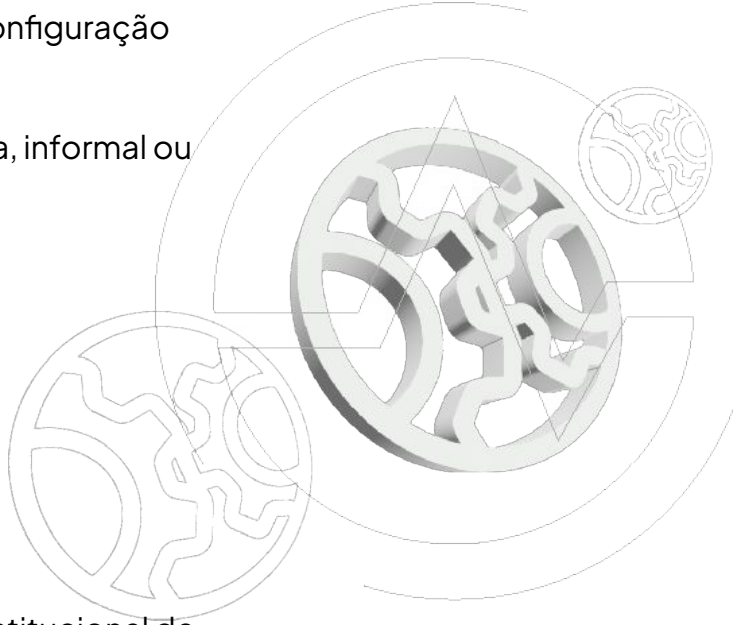
Isso significa que a progressão entre ambientes não é implícita, informal ou arbitrária.

Ela é codificada pela plataforma.

O pipeline pode expressar:

- Regras de promoção;
- Quality gates;
- Caminhos paralelos;
- E integrações com processos organizacionais.

A ideia correta é: **DeploymentPipeline** transforma a política institucional de promoção em recurso da plataforma.



Cluster-scoped versus namespace-scoped

A distinção entre **cluster-scoped** e **namespace-scoped** é central no **OpenChoreo**.

Ela define se um recurso será:

- Compartilhado globalmente;
- Ou isolado para um namespace específico.

Use **cluster-scoped** quando o objetivo for compartilhar padrões e infraestrutura em toda a instalação.

Use **namespace-scoped** quando for necessário:

- Isolar;
- Customizar;
- Sobrescrever;
- Ou adaptar localmente esses padrões.

Isso não é detalhe de implementação. É parte do modelo de governança da plataforma.

O que costuma ser compartilhado e o que costuma ser local

No setup padrão do **OpenChoreo**, vários recursos de plataforma tendem a ser compartilhados globalmente como **cluster-scoped**, incluindo:

- ClusterDataPlane;
- ClusterWorkflowPlane;
- ClusterComponentType;
- ClusterTrait;
- ClusterWorkflow.

Já recursos como:

- Project;
- Environment;
- DeploymentPipeline;

permanecem **namespace-scoped** porque pertencem ao contexto daquele namespace.

Esse modelo reduz duplicação e, ao mesmo tempo, preserva espaço para customização local quando necessário.

O que isso significa

Em uma implantação madura, o esperado é que a instituição mantenha padrões globais compartilhados para todas as turmas e programas.

Esses padrões podem ser publicados como recursos **cluster-scoped**.

Quando uma disciplina, laboratório ou programa específico precisar de exceções controladas, a organização pode criar variantes **namespace-scoped**.

Isso permite combinar:

- Padronização institucional;
- Adaptação local;
- Segurança;
- Reprodutibilidade;
- E autosserviço governado.

É exatamente esse equilíbrio que torna o **OpenChoreo** adequado para uso acadêmico compartilhado em escala.

Erros conceituais a evitar

Erro 1: “Namespace é só um namespace comum do Kubernetes.”
Incompleto. No OpenChoreo, namespace é parte explícita da organização e governança do Control Plane.

Erro 2: “Environment é só dev, staging ou prod.”
Incorreto. Environment é um recurso de primeira classe ligado a infraestrutura e promoção.

Erro 3: “DeploymentPipeline é só uma lista linear de ambientes.”
Incorreto. Ele pode expressar regras, gates e caminhos mais complexos.

Erro 4: “ComponentType é a mesma coisa que Component.”
Incorreto. ComponentType é definido pela plataforma; Component é criado pelo desenvolvedor.

Erro 5: “Trait é um detalhe opcional sem papel arquitetural.”
Incorreto. Trait é o mecanismo formal de capacidades transversais.

Erro 6: “Cluster-scoped e namespace-scoped têm baixo impacto.”
Incorreto. Essa distinção define o modelo de compartilhamento e isolamento da governança.

Síntese

As abstrações de plataforma do **OpenChoreo** são a camada em que o time de plataforma define topologia, governança e padrões reutilizáveis.

Namespaces delimitam escopo administrativo.

Environments definem contextos de runtime e estágios de entrega.

DataPlanes e **WorkflowPlanes** representam a infraestrutura de execução e automação.

DeploymentPipelines controlam a progressão entre ambientes.

ComponentTypes definem padrões base de deploy.

Traits adicionam capacidades transversais.

Workflows padronizam automações.

A distinção entre **cluster-scoped** e **namespace-scoped** determina o que é compartilhado globalmente e o que é isolado localmente.

Módulo III: Workflows

Last Updated: April 2026

Build e delivery workflows no OpenChoreo

Este módulo explica como o **OpenChoreo** transforma código-fonte em workloads implantáveis.

Também mostra que o modelo de workflows da plataforma é mais amplo do que CI.

No **OpenChoreo**, builds de componentes, provisionamento, data pipelines e outras automações usam o mesmo modelo conceitual baseado em **Workflow** e **WorkflowRun**.

O objetivo deste módulo é ensinar build e delivery como automações governadas pela plataforma, e não como scripts soltos ou pipelines avulsos.

Mensagem central do módulo

No **OpenChoreo**, build e delivery não são modelados como scripts avulsos, mas como workflows governados pela plataforma.

Quando se usa o CI interno, o componente referencia um **workflow** aprovado, o **OpenChoreo** executa esse workflow no **Workflow Plane**, constrói a imagem e gera ou atualiza o **Workload**.

Quando se usa CI externo, o pipeline institucional constrói a imagem e chama a API do **OpenChoreo** para criar ou atualizar o **Workload**.

Em ambos os casos, o **OpenChoreo** continua responsável por deployment, scaling e observabilidade.

Visão geral do modelo de workflows

O **OpenChoreo** adota um modelo unificado de automação com dois recursos principais:

- **Workflow template** governado de automação.
- **WorkflowRun** execução concreta desse template com valores específicos de parâmetros.

Esse modelo cobre tanto builds de componentes quanto automações genéricas.

CI, no **OpenChoreo**, é um uso especializado de um mecanismo mais amplo de automação.

Workflow não é apenas pipeline de CI

No **OpenChoreo**, **Workflow** não deve ser entendido apenas como pipeline de build.

A documentação oficial descreve **Workflow** como um template de automação que pode ser usado para:

- Builds de componentes;
- Provisionamento de infraestrutura;
- Data pipelines;
- Testes end-to-end;
- Publicação de pacotes;
- E outras tarefas operacionais.

A formulação correta é: CI é um caso especial dentro de um modelo mais amplo de automação governada.

O papel do Workflow Plane

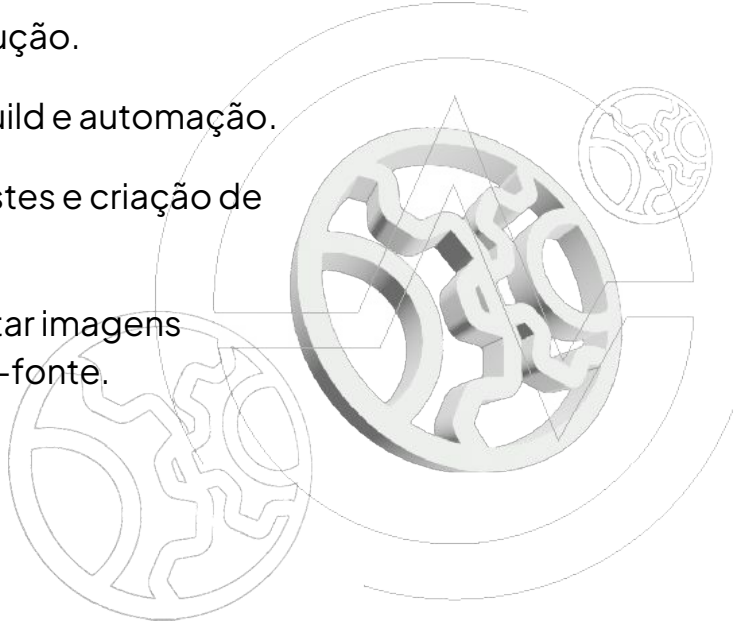
O **Workflow Plane** é o plano especializado em executar builds e workloads de automação.

O **Control Plane** hospeda os recursos e orquestra a execução.

O **Workflow Plane** executa as operações intensivas de build e automação.

Essa separação existe para impedir que compilações, testes e criação de imagens afetem diretamente o runtime das aplicações.

Sem **Workflow Plane**, o **OpenChoreo** ainda pode implantar imagens prontas, mas não executa build interno a partir de código-fonte.



O motor nativo de execução

Na implementação documentada, o **OpenChoreo** executa workflows com Argo Workflows.

WorkflowRuns atualmente suportam workflows baseados em Argo.

Isso significa que o desenho conceitual de **workflow** é amplo, mas o mecanismo nativo é Argo Workflows.

Esse ponto deve ser ensinado com precisão técnica.

Built-in workflow-based CI

No modelo **built-in workflow-based CI**, o componente é configurado para Build from Source.

Nesse fluxo:

- O **Component** referencia um **Workflow**;
- O **OpenChoreo** cria **WorkflowRuns**;
- O workflow constrói a imagem;
- E o sistema cria ou atualiza o **Workload**.

Esse é o modelo integrado de build do **OpenChoreo**.

O desenvolvedor não desenha o pipeline completo em baixo nível, ele seleciona um **workflow** permitido pela plataforma e fornece os parâmetros necessários.

O que o desenvolvedor realmente configura

No CI interno do **OpenChoreo**, o desenvolvedor não constrói livremente qualquer pipeline.

Ele configura o componente com:

- Um workflow aprovado;
- Parâmetros de build;
- Repositório Git;
- Branch;
- E, quando necessário, segredo para repositório privado.

A implementação real do **workflow** continua sob controle da plataforma.

A plataforma expõe ao desenvolvedor uma interface curada por schema, e não um pipeline arbitrário.

Quando um Workflow vira workflow de componente

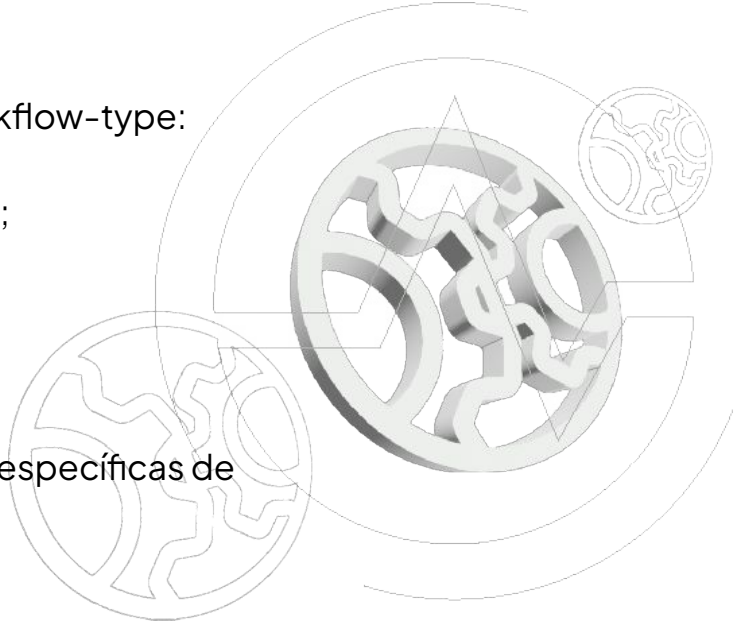
Um **Workflow** torna-se um workflow de componente quando atende às condições de governança da plataforma.

Na documentação atual, isso inclui:

- O workflow carregar o rótulo openchoreo.dev/workflow-type: "component";
- Um Component referenciá-lo em sua configuração;
- E o workflow estar listado em ComponentType.spec.allowedWorkflows.

Não existe uma CRD separada para “CI workflow”.

Trata-se do mesmo recurso **Workflow**, usado sob regras específicas de governança.



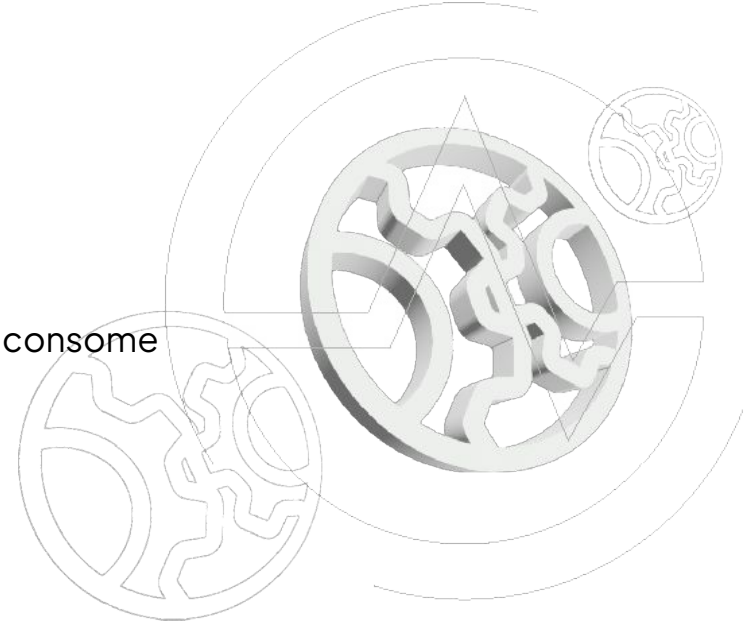
allowedWorkflows e governança

O campo allowedWorkflows do **ComponentType** define quais workflows um componente daquele tipo pode usar.

Isso permite que a equipe de plataforma imponha:

- Builders aprovados;
- Padrões de build;
- Requisitos de segurança;
- E políticas institucionais por tipo de componente.

O usuário não escolhe livremente qualquer pipeline, mas consome workflows autorizados pela plataforma.



Auto-build

O **OpenChoreo** suporta auto-build para componentes configurados com build a partir do código-fonte.

Nesse modelo, pushes no repositório Git podem disparar automaticamente novos builds.

O ponto importante é este: auto-build não substitui o **WorkflowRun**.

Ele automatiza a criação do **WorkflowRun** em resposta a eventos Git.

Assim, o modelo conceitual continua o mesmo; o que muda é o gatilho da execução.

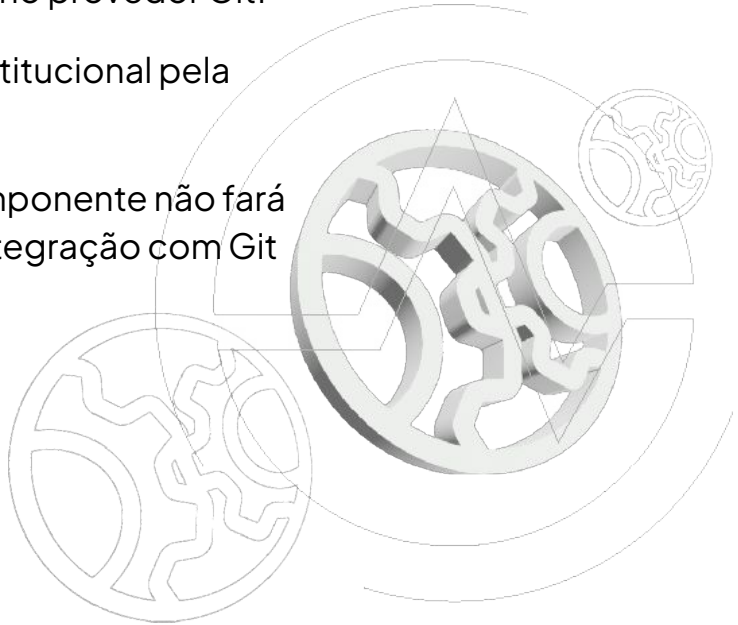
Auto-build depende de setup institucional

Auto-build não é apenas uma flag do lado do desenvolvedor.

A documentação oficial exige configuração de webhook no provedor Git.

Isso significa que **auto-build** depende de preparação institucional pela equipe de plataforma.

Em contexto acadêmico, esse ponto é essencial: um componente não fará build automático apenas por existir; a infraestrutura de integração com Git precisa estar corretamente configurada.



O que o build produz no OpenChoreo

No **OpenChoreo**, o resultado formal do build não é um Deployment direto.

Depois de construir e publicar a imagem, o workflow cria ou atualiza o **Workload** via **API do OpenChoreo**.

Esse passo usa autenticação OAuth 2.0 client credentials e chama a **Workload API** para criar ou atualizar o recurso.

O **Workload** então entra no lifecycle normal da plataforma para deployment e operação.

Como o Workload pode ser gerado

A documentação oficial descreve dois padrões principais para geração de **Workload** após o build:

1. A partir de um descritor vindo do repositório.
2. Por geração automática baseada, no mínimo, na imagem construída e nos metadados necessários.

Isso significa que o **OpenChoreo** pode trabalhar tanto com descrição explícita de workload quanto com geração automatizada no workflow de CI.

External CI

O **OpenChoreo** também suporta integração oficial com CI externo.

Nesse modelo:

- Um pipeline externo constrói a imagem;
- O pipeline chama a Workload API do **OpenChoreo**;
- O **Workload** é criado ou atualizado;
- E o **OpenChoreo** continua cuidando de deployment, scaling e observabilidade.

Isso permite integrar Jenkins, GitHub Actions, GitLab CI e outros sistemas sem abandonar o modelo operacional do OpenChoreo.

Por que um componente pode existir sem Workload

No modo External CI, o componente pode ser criado sem workload inicial.

A documentação oficial afirma isso explicitamente.

Nesse modelo, o componente representa a intenção e o enquadramento governado dentro da plataforma.

O **Workload** só aparece depois, quando o pipeline externo conclui o build e publica o workload via API.

Essa é a explicação correta para um dos comportamentos que mais geram dúvida em laboratório.

Quando usar built-in CI e quando usar External CI

Use **built-in CI** quando a instituição deseja uma experiência mais integrada de build e deploy no **OpenChoreo**, com menor esforço de integração e maior visibilidade nativa do processo.

Use **External CI** quando a organização já possui uma esteira institucional consolidada, ou quando deseja manter o build fora da plataforma e usar o **OpenChoreo** principalmente para deployment e operação.

A escolha entre os dois modelos deve ser tratada como decisão de governança e operação, não apenas de conveniência individual do desenvolvedor.

Síntese

No **OpenChoreo**, a transformação de código-fonte em **Workload** é governada por **workflows**.

No modo built-in, um componente referencia um workflow aprovado, o **OpenChoreo** executa esse workflow no **Workflow Plane**, constrói a imagem e cria ou atualiza o **Workload** via API.

No modo External CI, a instituição pode usar seu CI existente para construir a imagem e publicar o **Workload** no **OpenChoreo**.

Em ambos os casos, a plataforma continua responsável por deployment, scaling e observabilidade.

Workflow é mais amplo do que CI, e a governança sobre quais workflows podem ser usados fica com a plataforma.

Módulo IV: Abstrações de Desenvolvedor

Last Updated: April 2026

Conceitos do OpenChoreo: abstrações de desenvolvedor

Este módulo apresenta as abstrações com as quais o desenvolvedor trabalha diretamente no **OpenChoreo**.

O objetivo não é ensinar o aprendiz a começar por Deployment, Service, Ingress, NetworkPolicy ou manifests de infraestrutura.

O objetivo é ensinar o aprendiz a declarar intenção de aplicação por meio das abstrações da plataforma.

No OpenChoreo, o desenvolvedor modela:

- O domínio da aplicação;
- As unidades implantáveis;
- O contrato de runtime;
- As interfaces expostas;
- As dependências;
- As capacidades transversais;
- E, quando necessário, o processo de build.

Mensagem central do módulo

No **OpenChoreo**, o desenvolvedor não descreve infraestrutura bruta.

- Ele descreve intenção de aplicação.
- O **Project** delimita o contexto da aplicação.
- O **Component** representa a unidade implantável governada.
- O **Workload** expressa o que essa unidade precisa para executar.
- Os **Endpoints** e **Dependencies** explicitam conectividade e relações entre serviços.
- Os **Traits** adicionam capacidades transversais.
- O **Workflow**, quando usado, define como o código-fonte se transforma em artefato executável.

Hierarquia das abstrações do desenvolvedor

O **OpenChoreo** organiza aplicações em uma hierarquia simples:

- Namespace → Project → Component → Workload

- **Namespace:** fronteira da equipe ou organização, normalmente administrada pelo time de plataforma.
- **Project:** agrupamento lógico de componentes relacionados, associado a um deployment pipeline.
- **Component:** unidade implantável individual, como serviço, aplicação web ou tarefa agendada.
- **Workload:** especificação de runtime do componente, incluindo imagem, endpoints, variáveis de ambiente e dependências.

Project: a principal fronteira de domínio

Project é a principal unidade organizacional do desenvolvedor no **OpenChoreo**.

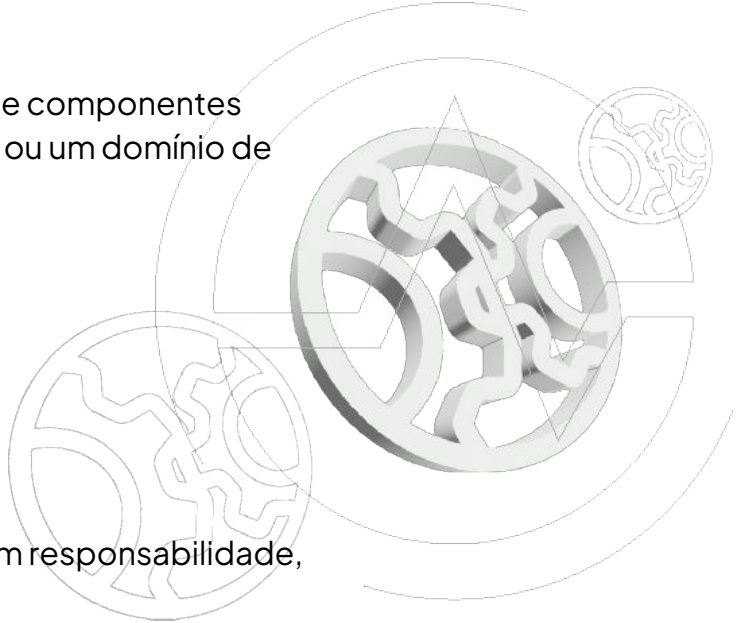
A documentação o define como um bounded context em termos de Domain-Driven Design.

Isso significa que um **Project** representa uma coleção coesa de componentes que, em conjunto, implementam uma capacidade de negócio ou um domínio de aplicação.

Project não deve ser tratado como:

- Pasta lógica;
- Etiqueta visual;
- Agrupamento arbitrário;
- Ou “um projeto por microserviço”.

Ele deve ser usado para delimitar um domínio de aplicação com responsabilidade, coesão e governança claras.



Project em runtime: de contexto lógico a fronteira operacional

No **OpenChoreo**, a fronteira declarada pelo **Project** não é apenas conceitual.

Ela se reflete no runtime por meio da arquitetura de células.

A documentação descreve que:

- Componentes do mesmo contexto compartilham uma fronteira operacional;
- A comunicação interna dentro dessa fronteira é tratada de forma natural;
- A comunicação entre fronteiras diferentes passa por caminhos definidos e governados.

Isso reforça um princípio pedagógico importante: o que o estudante declara como limite de domínio deve corresponder a uma fronteira real de operação, segurança e comunicação.

Como contextualizar Project em uma capacitação

Em contexto universitário, o docente deve orientar os estudantes a modelar um **Project** como um domínio de aplicação.

Exemplos corretos:

- Sistema de gestão acadêmica;
- Plataforma de reservas;
- Sistema de IoT de laboratório;
- Portal de apoio ao ensino.

Cada um desses **Projects** pode conter múltiplos componentes relacionados.

Antipadrão comum: criar um **Project** por microserviço.

O modelo correto é: um **Project** agrupa componentes relacionados de um mesmo domínio; os **Components** representam as unidades individuais dentro desse domínio.

Component: a unidade implantável governada

Component é a unidade implantável fundamental do **OpenChoreo**.

Mas **Component** não é apenas:

- Um repositório;
- Um container;
- Um microserviço;
- Ou um pedaço de código.

Um **Component** é uma entidade governada que conecta:

- Um **ComponentType** definido pela plataforma;
- Parâmetros específicos da aplicação;
- **Traits** para composição de capacidades;
- E, opcionalmente, **Workflow** de build.

Em outras palavras: o **Component** é a interface entre a intenção do desenvolvedor e os padrões controlados pelo time de plataforma.

ComponentType, parameters e traits

O **Component** referencia um ComponentType definido pela plataforma.

Esse **ComponentType** determina:

- O tipo de workload subjacente;
- Os parâmetros disponíveis;
- E quais recursos Kubernetes serão gerados.

Os **parameters** representam os valores específicos daquele componente.

Os **traits** representam capacidades transversais reutilizáveis que podem ser compostas no componente.

Exemplos de traits:

- Regras de alerting;
- OAuth2 proxy;
- Armazenamento persistente;
- Caching;
- Monitoramento.

O estudante não escolhe “qualquer forma” de deploy. Ele trabalha dentro de contratos definidos e governados pela plataforma.

Workload: o contrato de runtime

Workload é a especificação de runtime de um **Component** no **OpenChoreo**.

A documentação o define como o recurso que descreve:

- A configuração do container;
- Os endpoints de rede;
- E as dependências em outros serviços.

O **Workload** representa o contrato de runtime do componente.

Ele informa:

- Qual imagem será executada;
- Quais interfaces serão expostas;
- E quais dependências precisam ser injetadas.

O **Workload** não deve ser ensinado como “o Deployment do Kubernetes com outro nome” Ele deve ser ensinado como a declaração do que a aplicação precisa para executar.

Container dentro do Workload

Dentro do **Workload**, o bloco container define o que será executado.

Ele inclui:

- Imagem do container;
- Comando;
- Argumentos;
- Variáveis de ambiente;
- Arquivos e configurações associadas.

Esta é a parte do **Workload** que responde à pergunta: “o que roda?”

Mas esta não é a área para modelar toda a política operacional do sistema.

Políticas de infraestrutura, padrões de operação e mecanismos organizacionais pertencem aos ComponentTypes, **Traits** e mecanismos de binding definidos pela plataforma.

Endpoints: contrato de interface, não apenas porta

Endpoints especificam as interfaces de rede que o componente expõe.

Eles definem como o componente pode ser acessado e por quem.

Mais importante: um **endpoint** não é apenas porta e protocolo.

A documentação trata endpoint como relação contratual de interface.

Isso significa que o endpoint expressa:

- Protocolo;
- Comportamento esperado;
- Compatibilidade de API;
- Forma de exposição da aplicação.

A ideia correta é: **endpoint** é contrato exposto pela aplicação, não apenas porta aberta em runtime.

Dependencies: relações explícitas entre componentes e serviços

Dependencies tornam explícitas as relações entre componentes e serviços.

Quando um componente declara uma dependência, a plataforma pode:

- Resolver o endpoint de destino;
- Injetar detalhes de conexão no componente consumidor;
- Configurar políticas de rede;
- Rastrear a arquitetura da aplicação;
- Proteger e monitorar a comunicação.

No **OpenChoreo**, dependência não é apenas URL em variável de ambiente.

Ela é uma relação formal que a plataforma entende e governa.

Isso permite que a arquitetura da aplicação seja observável, rastreável e operacionalmente consistente.

Traits: capacidades anexáveis, composáveis e reutilizáveis

Traits são capacidades transversais que podem ser anexadas a um **Component**.

Eles são:

- Anexáveis;
- Composáveis;
- Reutilizáveis;
- Orientados a schema.

Um mesmo componente pode receber vários traits.

O mesmo **trait** pode ser reutilizado em vários componentes.

Um mesmo **trait** também pode ser instanciado múltiplas vezes, com nomes de instância e parâmetros próprios.

Traits não substituem o ComponentType.

Eles complementam o componente com capacidades adicionais de forma declarativa e governada.

Workflow configuration: build a partir do código-fonte

Workflow configuration define, opcionalmente, como construir o componente a partir do código-fonte.

Quando esse modelo é usado:

- O **Component** referencia um Workflow;
- O **OpenChoreo** executa um WorkflowRun;
- O **build** produz a imagem;
- O **Workload** pode ser criado automaticamente a partir da saída do build.

Esse mecanismo permite que o desenvolvedor declare como o código deve ser construído, sem precisar desenhar manualmente um pipeline de baixo nível.

O WorkflowRun é a execução efetiva do workflow.

Ele mantém rastreabilidade entre componente, build, commit e artefato produzido.

Dois padrões de uso: imagem pronta ou build gerenciado

O **OpenChoreo** suporta dois padrões principais de implantação:

1. A partir de imagem pré-construída: O desenvolvedor fornece a imagem diretamente no **Workload**. Aqui, não há etapa de build gerenciada pela plataforma.
2. A partir de código-fonte: O **Component** referencia um **Workflow** e o **OpenChoreo** executa o build, gera o artefato e cria o **Workload**.

Isso significa que **workflow** é opcional.

Nem todo componente precisa, necessariamente, de build gerenciado pelo **OpenChoreo**.

Esse ponto é especialmente importante em organizações que já utilizam GitHub Actions, GitLab CI ou Jenkins.

O que dizer

No **OpenChoreo**, vocês não começam modelando **Deployments** e **Services**.

Vocês começam modelando a aplicação:

- Qual é o domínio;
- Quais são os componentes;
- O que cada componente precisa para executar;
- Quais interfaces ele expõe;
- De quais serviços depende;
- E quais capacidades adicionais ele requer.

A plataforma traduz isso para a infraestrutura correta.

Esse é o modelo mental certo para usar a plataforma com maturidade arquitetural.

Erros conceituais a evitar

Erro 1: “Project é só uma pasta lógica.”
Incorreto. Project é a principal fronteira de domínio e organização da aplicação.

Erro 2: “Component é só o microserviço.”
Incorreto. Component é uma unidade implantável governada, composta por template, parâmetros, traits e, opcionalmente, workflow.

Erro 3: “Workload é só o Deployment com outro nome.”
Incorreto. Workload define o contrato de runtime da aplicação.

Erro 4: “Endpoint é só a porta.”
Incorreto. Endpoint é um contrato de interface.

Erro 5: “Dependency é só uma URL.”
Incorreto. Dependency é uma relação formal entre componentes e serviços.

Erro 6: “Todo componente precisa de workflow.”
Incorreto. A plataforma também suporta implantação com imagem pré-construída.

Síntese

As abstrações de desenvolvedor do **OpenChoreo** formam um modelo declarativo para construir e operar aplicações sem expor diretamente a complexidade da infraestrutura.

- O **Project** delimita o domínio da aplicação.
- O **Component** é a unidade implantável governada.
- O **Workload** define o contrato de runtime por meio de container, endpoints e dependencies.
- Os **Traits** compõem capacidades transversais reutilizáveis.
- O **Workflow configuration**, quando presente, define como o código-fonte é construído e convertido em workload implantável.

Essa é a forma correta de ensinar **OpenChoreo** como plataforma de modelagem e operação de aplicações.

Módulo V: Ciclo de Vida

Last Updated: April 2026

Da definição ao sistema em execução

Este módulo explica como o **OpenChoreo** converte definições declarativas em implantação real.

No **OpenChoreo**, o componente declarado pelo desenvolvedor não é implantado diretamente.

A plataforma usa uma cadeia explícita de recursos para transformar intenção em execução:

Component → ComponentRelease → ReleaseBinding → RenderedRelease

Essa cadeia existe para garantir:

- Governança;
- Rastreabilidade;
- Reprodutibilidade;
- Rollback;
- E promoção segura entre ambientes.

Mensagem central do módulo

No **OpenChoreo**, um componente declarado pelo desenvolvedor não é implantado diretamente.

Primeiro, seu estado é congelado em um **ComponentRelease** imutável.

Depois, esse release é vinculado a um ambiente por meio de um **ReleaseBinding**, que injeta apenas as diferenças específicas daquele ambiente.

Em seguida, o **OpenChoreo** rendeiriza os templates e gera os recursos Kubernetes finais, aplicando-os ao plano de execução.

Essa é a lógica correta do lifecycle da plataforma.

Visão geral do ciclo de vida

A cadeia central de lifecycle do **OpenChoreo** é:

Component → ComponentRelease → ReleaseBinding → RenderedRelease

Nessa cadeia:

- O **Component** expressa a intenção original;
- O **ComponentRelease** congela o estado implantável;
- O **ReleaseBinding** faz a ligação com um ambiente específico;
- O **RenderedRelease** contém os recursos Kubernetes finais.

Esse modelo separa claramente:

- Definição da aplicação;
- Versão promovível;
- Diferenças por ambiente;
- E materialização final no runtime.

O ponto de partida: o Component

O **Component** é a origem da intenção declarada do desenvolvedor.

Ele declara a aplicação e a vincula a um template definido pela plataforma.

O **Component**:

- Referencia um **ComponentType**;
- Pode usar **Traits**;
- Carrega parâmetros;
- E se relaciona com um **Workload**.

Mas o **Component** ainda não é o artefato final implantado, ele é o recurso de origem cuja configuração será congelada e depois transformada em execução.

O que o Component não é

É importante ensinar com rigor o que o **Component** não é.

O **Component** não é:

- O Deployment final;
- O release promovido entre ambientes;
- O binding por ambiente;
- Nem o conjunto final de objetos Kubernetes.

O **Component** está no nível da intenção declarativa.

Aquilo que será promovido entre ambientes é o seu congelamento formal: o **ComponentRelease**.

ComponentRelease: snapshot imutável

O **ComponentRelease** é um snapshot imutável da configuração do componente em um ponto específico no tempo.

Ele captura:

- O ComponentType;
- Os Traits;
- Os parâmetros;
- E o Workload com a imagem construída.

A documentação oficial destaca que esse recurso existe para garantir:

- Reprodutibilidade;
- Rollback;
- Auditoria;
- E promoção segura entre ambientes.

Didaticamente, o **ComponentRelease** funciona como um lock file da implantação.

O que exatamente é congelado no ComponentRelease

No **OpenChoreo**, a imutabilidade do release não é apenas um conceito abstrato.

O **ComponentRelease** congela:

- Owner, com vínculo a projeto e componente;
- ComponentType, como snapshot do template de plataforma;
- Traits, como snapshot das capacidades anexadas;
- ComponentProfile, com parâmetros e configurações de traits;
- Workload, como snapshot da especificação com a imagem construída.

Em outras palavras, o release congela o estado completo que será promovido.

Por que a imutabilidade importa

A imutabilidade do **ComponentRelease** é o que sustenta:

- Reprodutibilidade;
- Rollback confiável;
- Rastreabilidade;
- E integridade de entrega.

Se a release é imutável, a unidade promovida permanece estável ao atravessar ambientes.

Isso permite explicar corretamente:

- Por que a entrega precisa ser rastreável;
- Por que configurações não devem mudar silenciosamente;
- E por que rollback depende de um snapshot fiel do que foi implantado.

O que acontece quando algo muda no upstream

No **OpenChoreo**, uma mudança em recursos upstream não altera silenciosamente um release já existente.

Quando recursos como **ComponentType** ou configurações do componente mudam, a plataforma reconcilia essas relações e gera um novo **ComponentRelease**.

Como os releases antigos permanecem imutáveis, implantações já existentes não mudam automaticamente.

A mudança só entra em execução quando um novo release é efetivamente implantado.

Isso preserva governança sem introduzir efeito colateral destrutivo.

ReleaseBinding: vínculo de implantação por ambiente

ReleaseBinding é o recurso que conecta um **ComponentRelease** a um **Environment** específico.

Ele é o que efetivamente implanta um release em um **data plane**.

Mas o **ReleaseBinding** é mais do que uma simples referência; é nele que entram as diferenças específicas daquele ambiente.

O release continua imutável, e as variações por ambiente pertencem ao **binding**.

Onde entram as diferenças entre dev, staging e prod

No **OpenChoreo**, diferenças entre ambientes não devem ser implementadas como cópias manuais de manifests separados.

Elas pertencem aos **environmentConfigs** aplicados por **ReleaseBinding**.

Isso inclui, por exemplo:

- Escalabilidade em produção versus desenvolvimento;
- Limites de recursos;
- Storage classes;
- Políticas específicas de ambiente.

A lógica correta é: a **release** permanece igual; o **binding** aplica apenas as diferenças permitidas por ambiente.

ReleaseBinding não é apenas um ponteiro

A documentação oficial é explícita: o **ReleaseBinding** é mais do que uma referência.

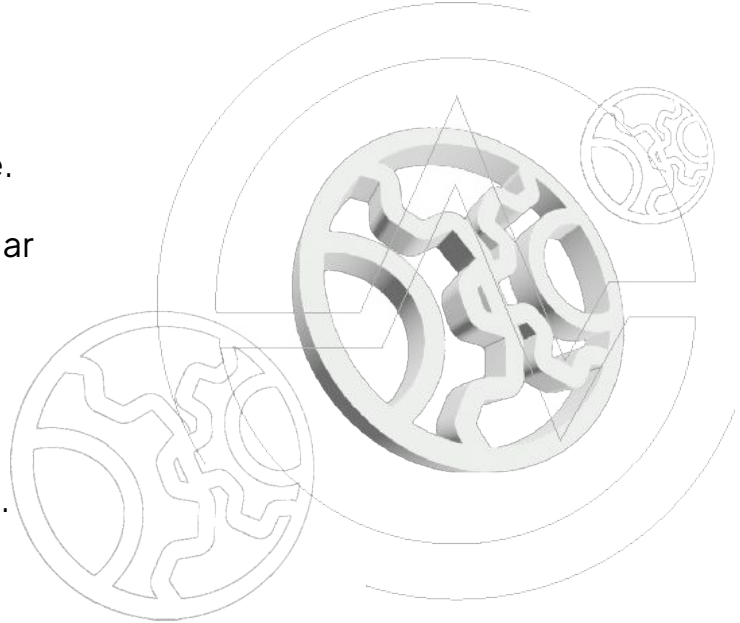
Ele combina:

- O snapshot imutável do release;
- Com as variações permitidas para aquele ambiente.

Esse modelo desloca as diferenças ambientais para o lugar conceitualmente correto.

Assim, o **OpenChoreo** evita dois problemas comuns:

- Drift entre ambientes;
- E edição manual divergente do artefato promovido.



Renderização e geração dos recursos finais

Depois que o **ReleaseBinding** é criado, o controlador de **ReleaseBinding** entra em ação.

Ele:

- Rendeiriza os templates de **ComponentType** e **Trait**;
- Combina a configuração congelada com os overrides do ambiente;
- Gera os recursos Kubernetes necessários;
- E produz um **RenderedRelease**.

O **RenderedRelease** contém o conjunto final de objetos Kubernetes que precisam ser aplicados ao plano-alvo.

O que é um RenderedRelease

O **RenderedRelease** representa a implantação real dos recursos da aplicação em um target plane.

Ele é criado pelo controlador de **ReleaseBinding**.

Seu papel é conter a lista completa de recursos Kubernetes finais que serão aplicados ao plano de destino.

Didaticamente, o **RenderedRelease** pode ser entendido como o pacote operacional final da implantação, já resolvido a partir de:

- Template institucional;
- Configuração congelada do release;
- E diferenças específicas do ambiente.

Os recursos Kubernetes que aparecem no final

O **OpenChoreo** não elimina os objetos clássicos do Kubernetes.

Ele os gera a partir da cadeia de abstrações de alto nível.

A documentação oficial mostra exemplos de recursos finais como:

- Deployment;
- Service;
- HTTPRoute;
- ConfigMaps;
- e outros objetos necessários à implantação.

Isso significa que o **OpenChoreo** não substitui o Kubernetes. Ele governa como esses recursos são produzidos.

Onde esses recursos reais vivem

Os recursos reais de Kubernetes não vivem no mesmo namespace do control plane onde o **Component** foi criado.

Eles são aplicados ao plano de execução, isto é, ao **data plane**.

Essa separação é intencional.

O **control plane** gerencia intenção, governança e reconciliação.

O **data plane** executa os **workloads** reais.

Esse é o motivo pelo qual um responsável pode criar um **Component** em um **namespace** lógico da plataforma e encontrar o **Deployment** real em um **namespace** do **data plane**.

Create once, deploy many

O **OpenChoreo** sustenta um modelo de entrega do tipo:

create once, deploy many

Isso significa que o mesmo **ComponentRelease** pode ser promovido com segurança através de múltiplos ambientes.

A unidade promovida é o release imutável, e não o **Component** mutável.

As diferenças entre ambientes entram apenas por meio dos **ReleaseBindings**.

Esse modelo reduz drift e melhora a confiabilidade da progressão entre dev, staging e produção.

Por que isso importa

Em ambiente acadêmico, este modelo permite ensinar uma disciplina correta de engenharia de software.

Ele ajuda a explicar por que:

- O mesmo projeto do aluno deve se comportar de forma consistente entre ambientes;
- A promoção para banca, homologação ou produção acadêmica não deve depender de ajustes manuais de última hora;
- Discrepâncias entre ambientes precisam ser declaradas e rastreáveis.

O **OpenChoreo** transforma essas boas práticas em comportamento nativo da plataforma.

Relações de criação, atualização e deleção

O lifecycle do **OpenChoreo** também possui dependências formais de criação, atualização e limpeza.

Exemplos:

- **DataPlanes** precisam existir antes que **Environments** possam apontar para eles;
- **Projects** precisam existir antes que **Components** possam ser criados;
- Mudanças em recursos upstream geram novos releases, em vez de mutar implantações já existentes;
- A deleção de um componente aciona limpeza em cascata dos recursos relacionados.

Isso mostra que o lifecycle é governado por relações explícitas entre recursos.

Erros conceituais a evitar

Erro 1: “O Component é implantado diretamente.”
Incorreto. Existe uma cadeia intermediária explícita: ComponentRelease, ReleaseBinding e RenderedRelease.

Erro 2: “ComponentRelease é só uma cópia do Component com outro nome.”
Incorreto. Ele é um snapshot imutável completo da configuração implantável.

Erro 3: “As diferenças entre ambientes devem ficar em arquivos separados por ambiente.”
Incorreto. Elas pertencem aos environmentConfigs aplicados por ReleaseBinding.

Erro 4: “O release muda à medida que passa pelos ambientes.”
Incorreto. O release permanece imutável.

Erro 5: “Deployment e Service ficam no mesmo namespace do Component.”
Incorreto. Eles são aplicados ao data plane.

Síntese

O **OpenChoreo** não implanta diretamente a definição mutável do componente.

Primeiro, ele congela essa definição em um **ComponentRelease** imutável.

Depois, um **ReleaseBinding** conecta esse release a um **Environment** e aplica somente as diferenças específicas daquele ambiente.

Em seguida, o controlador de **ReleaseBinding** rendeiriza os templates e produz os recursos Kubernetes finais, representados por um **RenderedRelease** e aplicados ao data plane.

Esse modelo sustenta:

- Reprodutibilidade;
- Rollback;
- Promoção segura;
- E o princípio de create once, deploy many.

Módulo VI: Customizações

Last Updated: April 2026

Customização da plataforma: visão geral

Este módulo explica como o **OpenChoreo** permite que a equipe de plataforma defina padrões institucionais reutilizáveis sem expor os estudantes diretamente à complexidade de Kubernetes.

No **OpenChoreo**, a customização da plataforma se organiza principalmente em torno de:

- **ComponentTypes**;
- **Traits**;
- templating com CEL;
- validações;
- e patching de recursos gerados.

O objetivo é oferecer liberdade segura: a organização define os caminhos aprovados, e os estudantes consomem esses caminhos sem precisar editar YAML bruto de Kubernetes.

Mensagem central do módulo

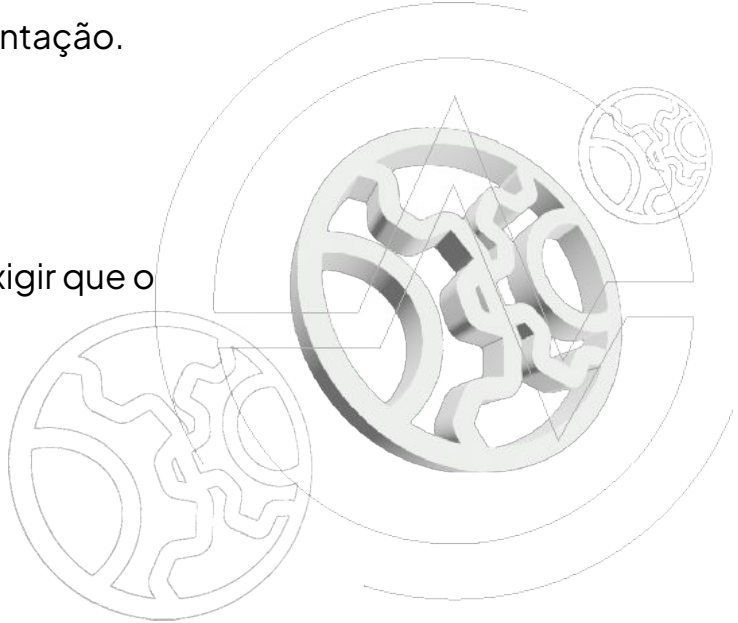
No **OpenChoreo**, a equipe de plataforma define o caminho seguro e reutilizável; o desenvolvedor apenas consome esse caminho.

ComponentTypes estabelecem o padrão-base de implantação.

Traits adicionam capacidades transversais.

CEL permite parametrização dinâmica.

O patching possibilita modificar recursos gerados sem exigir que o desenvolvedor edite YAML bruto de Kubernetes.



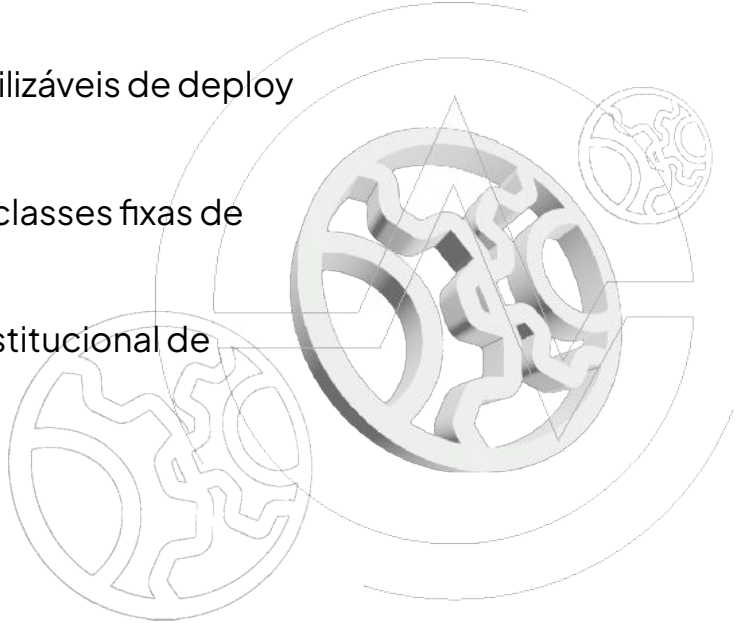
O que é um ComponentType

ComponentType é um recurso definido pela plataforma que determina como componentes são implantados e quais recursos serão gerados para eles.

Ele permite que a equipe de plataforma crie padrões reutilizáveis de deploy com parâmetros configuráveis.

No **OpenChoreo**, o **ComponentType** substitui a ideia de classes fixas de componentes por templates governados e extensíveis.

A formulação correta é: **ComponentType** é o template institucional de implantação.



O que um ComponentType controla

Um **ComponentType** pode definir:

- O workload principal;
- Os workflows permitidos;
- Os parâmetros expostos ao desenvolvedor;
- As configurações específicas por ambiente;
- Traits embutidos;
- Traits permitidos;
- Validações;
- E os templates de recursos Kubernetes que serão gerados.

Isso significa que o **ComponentType** é o ponto central de governança do padrão-base de deploy.

Workload types e padrão-base de deploy

No **OpenChoreo**, o campo **workloadType** define o tipo principal de workload do **ComponentType**.

A documentação atual lista os tipos principais:

- Deployment;
- Statefulset;
- Cronjob;
- Job;
- Proxy.

Para **workload types** diferentes de proxy, ao menos um **ResourceTemplate** precisa ter id igual ao workloadType.

Esse é o recurso que ancora a materialização principal do componente.

ComponentTypes padrão do OpenChoreo

O **OpenChoreo** já fornece **ComponentTypes** padrão que servem como ponto de partida para a plataforma.

Os exemplos oficiais atuais destacam:

- Deployment/service;
- Deployment/web-application;
- Cronjob/scheduled-task.

Esses exemplos demonstram:

- Parâmetros configuráveis;
- Resources com geração dinâmica;
- Overrides por ambiente;
- E padrões reais de uso para aplicações web, serviços backend e tarefas agendadas.

ClusterComponentType versus ComponentType

ComponentType é namespace-scoped.

ClusterComponentType é a variante cluster-scoped.

Use **ClusterComponentType** quando a instituição quiser compartilhar o mesmo padrão entre vários namespaces.

Use **ComponentType** namespace-scoped quando houver necessidade de customização ou override local.

Um detalhe importante: **ClusterComponentTypes** só podem referenciar **ClusterTraits** e **ClusterWorkflows**.

O que é um Trait

Trait é uma preocupação transversal que pode ser anexada a componentes para adicionar capacidades adicionais.

Exemplos típicos incluem:

- Storage persistente;
- Observabilidade;
- Políticas de segurança;
- Sidecars;
- E integrações auxiliares.

Trait modifica ou estende os recursos gerados pelo **ComponentType** sem exigir mudança no template-base.

A formulação correta é: **ComponentType** define o padrão-base; **Trait** acrescenta capacidades compostas sobre esse padrão.

Traits embutidos e Traits permitidos

No **ComponentType**, o campo **traits** representa instâncias de traits pré-configuradas que são aplicadas automaticamente a todos os componentes daquele tipo.

Já o campo allowedTraits define quais traits adicionais os desenvolvedores podem anexar livremente aos seus componentes.

Isso permite à organização combinar:

- Capacidades obrigatórias embutidas;
- E capacidades opcionais controladas.

Esse é um mecanismo central de governança com flexibilidade controlada.

ClusterTrait versus Trait

Trait é namespace-scoped.

ClusterTrait é a variante cluster-scoped.

Use **ClusterTrait** quando a capacidade transversal precisar ser compartilhada entre vários namespaces.

Use **Trait** namespace-scoped quando houver necessidade de variação local.

Observação importante: **ClusterTrait** compartilha a mesma estrutura básica de parameters, environmentConfigs, creates e patches, mas não suporta validations.

Parameters versus environmentConfigs

No **OpenChoreo**, **parameters** e **environmentConfigs** têm funções diferentes.

Parameters: configuração estática exposta ao desenvolvedor.

environmentConfigs: configuração específica por ambiente, passível de override no momento do binding e do deploy.

A regra correta é: O que faz parte do perfil estável do componente pertence a **parameters**, e o que varia entre development, staging e production pertence a **environmentConfigs**.

openAPIV3Schema como contrato formal

Tanto **parameters** quanto **environmentConfigs** usam SchemaSection com openAPIV3Schema.

Isso significa que o **OpenChoreo** usa o formato padrão OpenAPI v3 JSON Schema para definir:

- Tipos;
- Defaults;
- Enums;
- Campos obrigatórios;
- Restrições estruturais.

O contrato de configuração do **OpenChoreo** é formal, validável e documentável.

Ele não depende de campos arbitrários ou convenções implícitas.

Templating com CEL

O **OpenChoreo** usa CEL, entre ${...}, como linguagem de geração dinâmica em templates.

CEL pode ser usado em:

- Resources de ComponentTypes;
- Creates de Traits;
- Values de patches;
- IncludeWhen;
- e forEach.

Isso significa que **CEL** não é um detalhe periférico. Ele é a linguagem central de parametrização e geração dinâmica da plataforma.

Como CEL aparece nos templates

Nos **templates** do **OpenChoreo**, **CEL** pode aparecer em três formatos principais:

- Valor isolado, preservando o tipo original;
- Interpolação dentro de strings;
- Chaves dinâmicas de mapas.

Além disso, o sistema fornece campos de controle de geração:

- includeWhen, para criação condicional;
- forEach, para geração repetida de múltiplos recursos.

A plataforma também oferece funções built-in como:

- oc_omit();
- oc_merge();
- oc_generate_name();
- oc_dns_label();
- oc_hash().

Variáveis de contexto no templating

Os templates de **ComponentTypes** e **Traits** são avaliados sobre contextos estruturados da plataforma.

Entre os contextos documentados, destacam-se:

- Metadata;
- Parameters;
- EnvironmentConfigs;
- Workload;
- E configurações resolvidas usadas no processo de rendering.

Isso significa que o **OpenChoreo** não faz interpolação textual ingênua, ele avalia expressões sobre contexto conhecido e tipado.

Validation rules com CEL

ComponentTypes e **Traits** namespace-scoped podem definir validation rules baseadas em **CEL**.

Cada validação possui:

- Uma regra CEL;
- E uma mensagem de erro correspondente.

Todas as regras precisam resultar em true para que o rendering prossiga.

Esse mecanismo complementa o schema estrutural com validações semânticas, como:

- Réplicas mínimas;
- Faixas válidas de portas;
- Ou combinações permitidas de campos.

Creates versus patches

Traits operam de duas formas principais:

Creates: geram novos recursos Kubernetes que não existem no **ComponentType** base.

Patches: modificam recursos já gerados pelo **ComponentType**.

Exemplos típicos:

- Um **trait** de storage pode criar PersistentVolumeClaims;
- Um **trait** de monitoramento pode criar ServiceMonitor;
- Um **trait** de sidecar pode alterar um Deployment já existente.

A distinção correta é: **creates** adiciona novos objetos; **patches** altera objetos existentes.

Patch model no OpenChoreo

O patching em **Traits** usa JSON Patch com extensões específicas do **OpenChoreo**.

A documentação atual destaca três extensões principais:

- Array filtering com sintaxe semelhante a JSONPath;
- CEL-based resource targeting;
- forEach iteration support.

As operações suportadas incluem:

- Add;
- Replace;
- Remove.

Isso torna o **patching** suficientemente expressivo para cenários reais de customização institucional.

O que o patching permite fazer

Com patching, um **Trait** pode:

- Injetar variáveis de ambiente;
- Adicionar volume mounts;
- Anexar sidecars;
- Modificar réplicas;
- Acrescentar labels e annotations;
- Ou remover partes do template base.

Esse é o mecanismo que permite customizar recursos gerados sem obrigar o engenheiro a editar o YAML bruto de Kubernetes.

Golden paths para ambiente de capacitação

Em ambiente acadêmico, o **OpenChoreo** permite construir golden paths governados.

A instituição pode oferecer, por exemplo:

- Um tipo padrão para serviços HTTP;
- Um tipo padrão para aplicações web;
- Um tipo padrão para tarefas agendadas;
- E traits aprovados para storage, sidecars, monitoramento, alertas e integrações auxiliares.

O resultado é liberdade segura: os estudantes desenvolvem projetos reais dentro de padrões aprovados institucionalmente.

Erros conceituais a evitar

Erro 1: “ComponentType é só um template visual do portal.”
Incorreto. Ele determina como componentes são implantados e quais recursos serão gerados.

Erro 2: “Trait é um detalhe acessório.”
Incorreto. Traits são a abstração formal de capacidades transversais.

Erro 3: “parameters e environmentConfigs são equivalentes.”
Incorreto. Um é configuração estática; o outro é configuração específica por ambiente.

Erro 4: “Patch no OpenChoreo é só JSON Patch puro.”
Incorreto. Há extensões de array filtering, targeting por CEL e iteração.

Erro 5: “CEL é detalhe avançado e raro.”
Incorreto. CEL é o núcleo do templating dinâmico da plataforma.

Síntese

No **OpenChoreo**, a customização de plataforma é construída em torno de **ComponentTypes** e **Traits**.

ComponentTypes definem os templates institucionais de implantação e os recursos que serão gerados.

Traits acrescentam capacidades transversais de forma composável.

Ambos usam schemas com **parameters** e **environmentConfigs**, templating com **CEL** e, no caso dos **Traits**, mecanismos de creates e patches para estender ou modificar os recursos base.

Esse modelo permite que a universidade ofereça caminhos seguros, reutilizáveis e expressivos, sem exigir que os estudantes operem diretamente YAML bruto de Kubernetes.

Módulo VII: Observabilidade

Last Updated: April 2026

Observabilidade, debugging e visibilidade operacional no OpenChoreo

Este módulo mostra como o **OpenChoreo** trata observabilidade como uma capacidade de plataforma de primeira classe.

No **OpenChoreo**, observabilidade não é um add-on secundário nem apenas um dashboard.

Ela é parte da arquitetura da plataforma e existe para centralizar:

- Logs;
- Métricas;
- Traces;
- Alertas;
- E incidentes.

O objetivo deste módulo é ensinar troubleshooting estruturado, e não apenas inspeção manual de pods e tentativa e erro.

Mensagem central do módulo

No **OpenChoreo**, observabilidade é um plano especializado da arquitetura.

O **Observability Plane** centraliza logs, métricas, traces, alertas e incidentes.

Esses dados ficam acessíveis por meio da Observer API, com filtros baseados nos próprios conceitos da plataforma, como:

- Componente;
- Projeto;
- Ambiente;
- E workflow run.

A forma correta de ensinar debugging no **OpenChoreo** é: observar, correlacionar, diagnosticar e então agir.

O que é o Observability Plane

O **Observability Plane** fornece a infraestrutura centralizada para coletar e analisar sinais operacionais da plataforma.

Seu papel é oferecer uma visão unificada:

- Do comportamento das aplicações;
- E da própria operação da plataforma.

A documentação oficial o trata como um plano arquitetural próprio, com integração a logs, métricas, traces, alerting e investigação operacional.

A formulação correta é: o **Observability Plane** transforma sinais técnicos dispersos em visibilidade operacional estruturada.

Single-cluster e multi-cluster

O **OpenChoreo** suporta topologias de observabilidade em single-cluster e multi-cluster.

Em **single-cluster**, todos os planos podem coexistir no mesmo cluster, o que simplifica aprendizado, laboratório e prova de conceito.

Em **multi-cluster,** o **Observability Plane** pode rodar em um cluster dedicado, enquanto **Data Planes** e **Workflow Plane** publicam telemetria remotamente para ele.

Essa distinção é importante porque muda o caminho de coleta e publicação da telemetria, mas preserva a experiência unificada de observabilidade.

Coleta em topologia multi-cluster

Na topologia **multi-cluster**, o **OpenChoreo** implanta coletores locais nos planos que geram telemetria.

A documentação atual descreve este modelo:

- O **Data Plane** implanta Fluent Bit para logs;
- Prometheus Agent para métricas;
- OpenTelemetry Collector para traces;
- E o **Workflow Plane** implanta Fluent Bit para logs de workflow run.

Todos esses coletores publicam dados por meio do gateway ingress do **Observability Plane**.

Pilha padrão de telemetria

O **OpenChoreo** já fornece uma pilha padrão coerente para observabilidade.

Por padrão:

- OpenSearch é usado para logs;
- OpenSearch é usado para traces;
- Prometheus é usado para métricas.

A arquitetura também é modular.

Isso significa que a Observer API permanece como interface estável, enquanto os backends podem ser substituídos ou suplementados por módulos compatíveis.

Logs

No **OpenChoreo**, logs são coletados dos containers do cluster e enriquecidos com metadados da plataforma.

Isso permite consultar logs por conceitos do **OpenChoreo**, como:

- Projeto;
- Componente;
- Ambiente;
- E, quando aplicável, workflow run.

Isso é pedagogicamente importante porque o estudante não precisa começar a investigação apenas por nome de pod ou namespace bruto de Kubernetes.

Métricas

As métricas no **OpenChoreo** fazem parte da mesma superfície unificada de observabilidade.

Elas permitem investigar sinais como:

- Consumo de CPU;
- Consumo de memória;
- Volume de tráfego;
- Latência;
- Taxa de erro;
- Ou ausência de requisições.

Nem todo problema aparece em logs. A plataforma trata métricas como parte formal do diagnóstico operacional, e não como ferramenta isolada.

Traces

Traces permitem acompanhar fluxos distribuídos de execução entre serviços e componentes.

No modelo documentado, o **Data Plane** usa OpenTelemetry Collector para coletar traces e publicá-los no **Observability Plane**.

Para que isso funcione, a aplicação precisa estar instrumentada para enviar traces via OTLP.

A formulação correta é: logs mostram eventos, métricas mostram comportamento agregado, e traces mostram encadeamento distribuído de execução.

Observer API

A Observer API é a interface canônica de observabilidade do **OpenChoreo**.

Ela unifica o acesso a:

- Logs;
- Métricas;
- Traces;
- Alertas;
- E incidentes persistidos.

Também lida com autenticação e autorização com base na identidade do usuário **OpenChoreo**.

Isso significa que observabilidade não é uma porta lateral insegura para os dados da plataforma.

Consultas por conceitos do OpenChoreo

Um dos pontos mais importantes do **OpenChoreo** é que a telemetria pode ser consultada usando os próprios conceitos da plataforma.

A documentação destaca filtros como:

- Projeto;
- Componente;
- Ambiente;
- E, no caso de logs, também workflow run.

Isso aproxima a observabilidade da forma como a aplicação é ensinada, modelada e operada.

Alerting

Alerting é fornecido pelo **Observability Plane** em integração com logs e métricas.

No **OpenChoreo**, alertas podem ser definidos por regras e disparar notificações por canais configurados, como email ou webhook.

Quando configurado, o sistema também pode persistir:

- Eventos de alerta;
- E incidentes.

Isso significa que o **OpenChoreo** não trata alertas apenas como mensagens efêmeras; ele os incorpora à memória operacional da plataforma.

Alertas, Traits e ReleaseBinding

No **OpenChoreo**, **alerting** pode ser integrado ao mesmo modelo declarativo usado em outros aspectos da plataforma.

A documentação atual descreve um fluxo em que:

- Platform engineers oferecem **traits** de regra de alerta;
- Desenvolvedores anexam esses **traits** aos componentes;
- E valores específicos por ambiente são configurados por meio de **environment configs** e **ReleaseBinding**.

Isso é muito importante didaticamente, porque conecta observabilidade aos módulos de **traits**, lifecycle e promoção entre ambientes.

RCA Agent com IA

O **RCA Agent** é um componente opcional com IA para análise de causa raiz.

A documentação descreve que ele analisa:

- Logs;
- Métricas;
- Traces

para gerar relatórios com causas prováveis de incidentes.

Ele pode ser acionado automaticamente quando incident creation e **AI RCA** estão habilitados na regra de alerta correspondente.

Como contextualizar o RCA Agent corretamente

O **RCA Agent** deve ser apresentado como apoio diagnóstico, e não como substituto do julgamento técnico humano.

A própria documentação alerta que a análise gerada por IA pode conter erros e recomenda verificar os achados antes de agir.

A formulação correta é: o **RCA Agent** acelera hipóteses de investigação, mas não substitui validação técnica.

Esse ponto é essencial em ambiente acadêmico.

Sequência pedagógica correta de debugging

A sequência mais útil para ensinar debugging no **OpenChoreo** é:

1. Verificar build logs quando a falha estiver no build.
2. Verificar runtime logs quando a aplicação sobe mas falha em execução.
3. Verificar métricas quando houver saturação, anomalias de uso ou ausência de tráfego.
4. Verificar traces quando o problema envolver fluxo distribuído ou latência entre serviços.
5. Verificar alertas e incidentes quando a falha já tiver sido capturada por regras.
6. Usar **AI RCA** como ferramenta auxiliar de investigação.

Esse é o caminho que transforma troubleshooting em prática estruturada.

Erros conceituais a evitar

Erro 1: “Observabilidade é só um dashboard.”
Incorreto. Ela é um plano especializado com coleta, análise, alerting e API unificada.

Erro 2: “Observer API serve só para logs.”
Incorreto. Ela unifica logs, métricas, traces, alertas e incidentes.

Erro 3: “Para investigar problemas, basta procurar nomes de pod.”
Incorreto. O OpenChoreo permite consulta por conceitos da própria plataforma.

Erro 4: “AI RCA dá a resposta final.”
Incorreto. Ele produz causas prováveis e exige validação humana.

Erro 5: “Alerting é uma função externa ao modelo declarativo.”
Incorreto. Ele pode ser integrado com traits, environment configs e ReleaseBinding.

Síntese

No **OpenChoreo**, observabilidade é um plano arquitetural próprio.

O **Observability Plane** centraliza logs, métricas, traces, alertas e incidentes.

A **Observer API** oferece uma interface unificada e autenticada para consulta desses dados.

A plataforma permite filtrar telemetria usando conceitos como componente, projeto, ambiente e workflow run.

Em topologias multi-cluster, os planos publicam telemetria para o **Observability Plane** por meio de coletores locais.

Sobre essa base, o **OpenChoreo** adiciona **alerting** e, opcionalmente, um **RCA Agent** com IA para apoiar a investigação de causa raiz.

Módulo VIII: Extensibilidade

Last Updated: April 2026

Extensibilidade e evolução avançada da plataforma no OpenChoreo

Este módulo explica como o **OpenChoreo** cresce além da instalação padrão por meio de uma arquitetura modular.

O **OpenChoreo** não foi desenhado como uma plataforma presa a um stack tecnológico único.

Ele foi projetado como uma **Internal Developer Platform** modular, com pontos de extensibilidade claros para integração de ferramentas em áreas como:

- API Gateway;
- CI;
- Observability;
- GitOps;
- e portal do desenvolvedor.

A instalação padrão oferece um caminho rápido para começar, mas a arquitetura permite evolução disciplinada quando a organização precisa.

Mensagem central do módulo

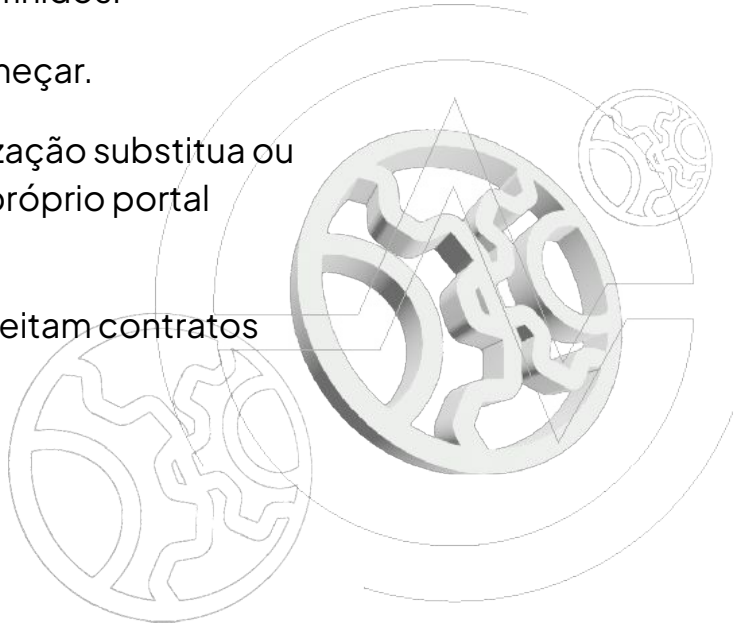
O **OpenChoreo** não é um produto preso a um stack tecnológico fixo.

Ele é uma plataforma com contratos de extensão bem definidos.

A instalação padrão oferece um caminho rápido para começar.

Mas a arquitetura foi projetada para permitir que a organização substitua ou complemente gateway, CI, observabilidade, GitOps e o próprio portal conforme suas necessidades.

A flexibilidade existe, mas depende de módulos que respeitam contratos formais de integração.



O que é um módulo no OpenChoreo

No **OpenChoreo**, módulos são integrações plugáveis que estendem capacidades da plataforma em pontos de extensibilidade definidos.

A documentação distingue dois tipos principais:

- OpenChoreo Modules;
- Curated Backstage Modules.

Esses dois tipos de módulo atendem a necessidades diferentes e não devem ser confundidos.

Essa distinção é central para compreender como a plataforma evolui de forma segura e coerente.

OpenChoreo Modules

OpenChoreo Modules estendem pontos de extensão nos planos da plataforma.

Eles se conectam aos seguintes planos:

- Data Plane;
- Workflow Plane;
- Observability Plane;
- Control Plane.

Esses módulos cobrem, principalmente, áreas como:

- API Gateway;
- CI;
- Observability;
- GitOps.

Em termos formais, um **OpenChoreo Module** é empacotado como Helm chart.

Definição formal de OpenChoreo Module

Um **OpenChoreo** Module é, formalmente, um Helm chart que:

- Instala a ferramenta subjacente no cluster Kubernetes-alvo;
- Conecta essa ferramenta aos contratos de integração do **OpenChoreo**;
- Fornece documentação para que operadores saibam como configurá-la e operá-la.

Essa definição é importante porque mostra que um módulo não é um plugin cosmético.

Ele é o mecanismo estrutural pelo qual uma tecnologia concreta passa a participar da plataforma.

Curated Backstage Modules

Curated Backstage Modules são plugins Backstage validados e incorporados ao portal do **OpenChoreo**.

Eles não são instalados separadamente como Helm charts.

Eles são compilados no portal.

Para adicioná-los, a equipe precisa:

- Fazer fork do portal Backstage do OpenChoreo;
- Integrar o plugin desejado;
- Construir uma imagem customizada do portal;
- E implantar essa imagem atualizada.

Essa é a principal diferença entre extensão dos planos da plataforma e extensão da experiência do portal.

Por que o OpenChoreo adota arquitetura modular

A documentação oficial apresenta três razões explícitas para a arquitetura modular do **OpenChoreo**:

1. Evitar vendor lock-in;
2. Permanecer enxuto por padrão e extensível sob demanda;
3. Estimular um ecossistema orientado pela comunidade.

Isso significa que o **OpenChoreo** foi desenhado para começar simples, mas evoluir quando a organização realmente precisar.

Defaults sensatos da instalação padrão

O **OpenChoreo** já entrega uma instalação padrão funcional com defaults sensatos.

A documentação atual cita como defaults:

- kgateway;
- Argo Workflows;
- OpenSearch;
- Flux CD.

Esses defaults existem para permitir adoção rápida e baixa complexidade inicial.

Eles são ponto de partida, e não imposição imutável.

API Gateway Modules

A área de **API Gateway** é uma das áreas formais de extensibilidade do **OpenChoreo**.

O requisito central documentado é: o módulo de gateway precisa fornecer uma implementação compatível com Kubernetes Gateway API.

Isso é necessário porque o **Control Plane** cria recursos Gateway e HTTPRoute e espera que um controlador compatível os realize.

A documentação também cita o kgateway como módulo padrão e menciona alternativas compatíveis, como Traefik, Cilium Gateway, Kong, Istio e Nginx Gateway Fabric.

O que um módulo de API Gateway precisa cumprir

Segundo a documentação, um módulo de **API Gateway** precisa:

- Instalar um GatewayClass;
- Aceitar recursos Gateway;
- Rotear tráfego com base em HTTPRoute;
- Ser instalado no namespace do Data Plane;
- E documentar aspectos como TLS, acesso externo e políticas de tráfego.

A formulação correta é: substituir o gateway é permitido, mas somente quando o módulo respeita o contrato Gateway API esperado pelo **OpenChoreo**.

GitOps Modules

A área de GitOps é responsável por continuous delivery e reconciliação de workloads a partir de configuração declarativa em Git.

O default documentado é Flux CD.

A documentação afirma que um módulo GitOps precisa:

- Instalar um motor de GitOps;
- Reconciliar manifests Kubernetes a partir de repositório Git;
- Permitir que o **OpenChoreo** configure repository, path, branch e interval;
- E expor status de sincronização para os operadores.

GitOps, no **OpenChoreo**, é uma área formal de extensibilidade, e não um detalhe invisível da instalação.

Observability Modules

A área de Observability também é formalmente extensível.

A documentação afirma que logs, traces e métricas podem usar módulos diferentes, inclusive combinando backends distintos.

Os defaults documentados incluem:

- OpenSearch e Fluent Bit para logs;
- OpenSearch e OpenTelemetry Collector para traces;
- Prometheus para métricas.

Isso permite manter uma experiência unificada de observabilidade sem prender a plataforma a um único backend fixo.

Adapter pattern em observabilidade

A documentação oficial afirma que módulos de observabilidade seguem um adapter pattern.

O Observer não conversa diretamente com cada backend.

Em vez disso, cada módulo fornece:

- Um backend;
- E um adapter que implementa um contrato OpenAPI estável entre o **OpenChoreo** e esse backend.

Esse padrão desacopla a Observer API das APIs nativas de cada ferramenta e permite trocar backends sem quebrar a superfície estável da plataforma.

CI Modules

CI também aparece na documentação como área formal de extensibilidade.

Conceitualmente, um módulo de CI deve:

- Instalar um workflow engine no workflow plane;
- Suportar o schema de workflow do OpenChoreo;
- E expor status de execução para os controladores.

Ao mesmo tempo, a documentação operacional atual é cuidadosa ao mostrar Argo Workflows como engine nativamente suportado no fluxo padrão.

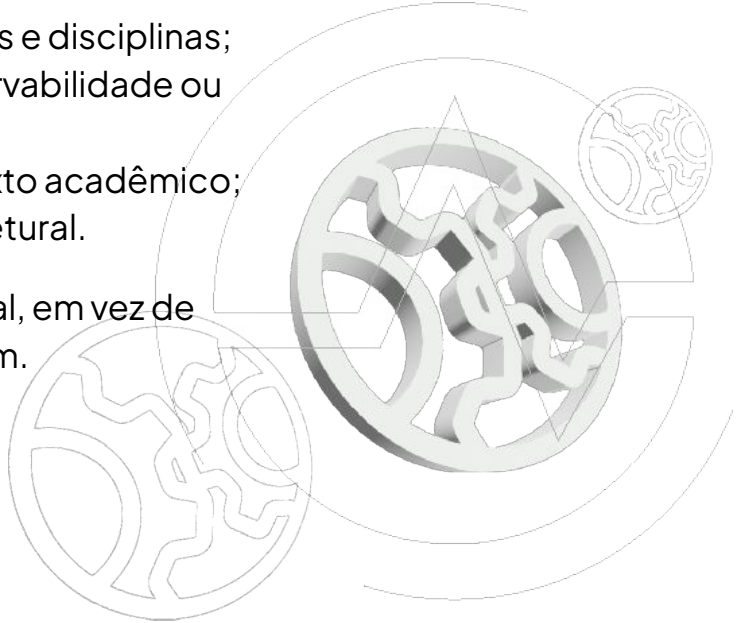
Portanto, a extensibilidade em CI existe, mas o grau de maturidade plug-and-play não é idêntico ao de todas as outras áreas.

O que isso significa

Para uma organização, a arquitetura modular do **OpenChoreo** significa que a instituição pode:

- Começar com a instalação padrão para laboratórios e disciplinas;
- Preservar investimentos prévios em gateway, observabilidade ou entrega contínua;
- Enriquecer o portal com plugins voltados ao contexto acadêmico;
- E evoluir a plataforma sem perder coerência arquitetural.

A plataforma pode crescer com a maturidade institucional, em vez de precisar ser substituída quando novas exigências surgirem.



Quando manter os defaults

A orientação mais equilibrada, consistente com a documentação, é manter os defaults quando:

- A instituição está começando com **OpenChoreo**;
- A prioridade é adoção rápida;
- A simplicidade operacional é mais importante do que customização;
- Não há exigência institucional forte por ferramentas específicas;
- O foco principal é ensino, e não customização profunda da plataforma.

A lógica correta é: começar simples e evoluir quando houver razão clara para isso.

Quando vale a pena estender a plataforma

Vale a pena estender a plataforma quando:

- A instituição já possui stack consolidado de gateway, observabilidade ou GitOps;
- Existem exigências institucionais por ferramenta específica;
- Há necessidade de escalar uma área da plataforma de forma diferente do default;
- Ou o portal do desenvolvedor precisa de plugins adicionais voltados ao contexto acadêmico.

A extensibilidade do **OpenChoreo** existe para acomodar necessidades reais, não para incentivar customização gratuita.

O que mencionar com mais cuidado

É importante ensinar com clareza que:

- **OpenChoreo Module** e **Curated Backstage Module** não são a mesma coisa;
- Módulo não significa integração arbitrária;
- Cada área possui contratos formais de integração;
- A modularidade não elimina os defaults;
- E estender a plataforma não significa personalizar tudo desde o primeiro dia.

Essa é a leitura mais fiel e mais madura da documentação oficial.

Erros conceituais a evitar

Erro 1: “OpenChoreo é preso a um stack fixo.”
Incorreto. A documentação enfatiza modularidade e pluggability.

Erro 2: “Curated Backstage Modules são instalados como Helm charts.”
Incorreto. Eles são plugins compilados no portal.

Erro 3: “Qualquer ferramenta pode ser acoplada de qualquer jeito.”
Incorreto. Cada área possui contratos formais de integração.

Erro 4; “Substituir defaults é sempre melhor.”
Incorreto. A documentação defende defaults sensatos e extensão sob demanda.

Erro 5: “CI é totalmente drop-in como qualquer outro módulo.”
Incorreto. A documentação atual é mais cuidadosa quanto à maturidade de CI no fluxo padrão.

Síntese

O **OpenChoreo** foi projetado como uma plataforma modular.

Ele entrega uma instalação padrão funcional e coerente, mas permite substituição ou complementação de gateway, CI, observabilidade, GitOps e portal por meio de módulos e contratos formais de integração.

OpenChoreo Modules estendem os planos da plataforma via Helm charts.

Curated Backstage Modules estendem o portal por meio de plugins compilados na imagem.

Esse modelo permite que a universidade comece simples e evolua a plataforma conforme suas necessidades, sem perder consistência arquitetural.

Módulo IX: Networking

Last Updated: April 2026

Networking, gateway e modelo de exposição no OpenChoreo

Este módulo explica como as aplicações são expostas e roteadas no **OpenChoreo**.

No **OpenChoreo**, exposição e roteamento não acontecem de forma mágica ou opaca.

O modelo correto é:

- O **Workload** declara endpoints;
- A plataforma gera recursos de rede;
- O **Data Plane** hospeda o runtime e a topologia de gateway;
- E o gateway encaminha o tráfego para o serviço correto.

A pergunta “onde minha aplicação está exposta?” deve ser respondida a partir do **Data Plane**, dos **endpoints** do **Workload** e dos recursos gerados pela plataforma.

Mensagem central do módulo

No **OpenChoreo**, a exposição de uma aplicação acontece no **Data Plane**.

O desenvolvedor declara endpoints no **Workload**.

A plataforma gera Service e HTTPRoute.

Um gateway compatível com Kubernetes Gateway API, configurado no Data Plane ou sobrescrito no Environment, recebe o tráfego e o encaminha para o serviço correto.

Essa é a explicação correta do modelo de exposição da plataforma.

O Data Plane é runtime e gateway ao mesmo tempo

O **Data Plane** não é apenas o local onde os workloads executam.

Ele também é o domínio em que:

- A topologia de gateway é aplicada;
- Políticas de rede são efetivadas;
- E o tráfego é roteado até os serviços.

Em outras palavras: o **Data Plane** é simultaneamente o ambiente de runtime e o ambiente de exposição da aplicação.

Separação entre Control Plane e Data Plane

No **OpenChoreo**, o **Control Plane** gerencia intenção, governança e reconciliação.

O **Data Plane** executa workloads e aplica os recursos reais de runtime e networking.

Por isso, os objetos Kubernetes subjacentes, como Deployment, Service e HTTPRoute, não aparecem no mesmo namespace em que o Component foi criado no Control Plane.

Eles são aplicados em namespaces do **Data Plane**.

Essa separação é intencional e faz parte da arquitetura da plataforma.

Endpoints no Workload

No **OpenChoreo**, endpoints descrevem as interfaces de rede que a aplicação expõe.

Cada endpoint declara:

- Protocolo;
- Porta;
- E o papel daquela interface no modelo da aplicação.

Endpoint não deve ser ensinado apenas como “uma porta aberta”. Ele é parte da intenção declarada do componente e serve de base para que a plataforma gere os recursos corretos de conectividade e exposição.

O que os endpoints habilitam

Quando um **endpoint** é declarado no **Workload**, a plataforma consegue inferir como conectar e expor aquela interface da aplicação.

Isso permite:

- Criação automática de Service;
- Geração de recursos de roteamento;
- E aplicação coerente de políticas de rede.

A ideia correta é: o **endpoint** é o contrato de rede da aplicação dentro do **OpenChoreo**.

URL pública e papel do nome do endpoint

No walkthrough oficial de deploy, a URL resultante segue a estrutura:

http://{environment}-{namespace}.{gateway-host}/{endpoint-name}/{path}

Isso mostra que o nome do **endpoint** participa diretamente do caminho exposto.

Portanto, **endpoint** não é um detalhe invisível da implementação.

Ele participa do contrato público de roteamento da aplicação.

O que a plataforma gera no deploy

Durante o deploy, o **OpenChoreo** gera recursos Kubernetes subjacentes a partir da cadeia declarativa da plataforma.

A documentação oficial mostra, entre eles:

- Deployment;
- Service;
- HTTPRoute.

Esses recursos são aplicados ao **namespace** do **Data Plane** correspondente ao ambiente de execução.

Isso significa que o desenvolvedor não precisa modelar esses objetos manualmente no fluxo padrão.

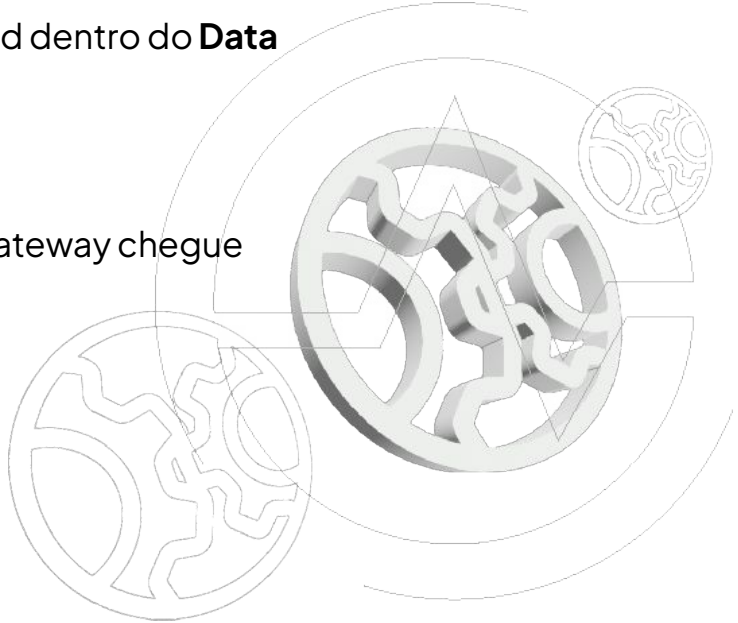
O papel do Service

No modelo do **OpenChoreo**, o **Service** fornece conectividade estável para os pods da aplicação.

Ele é o recurso que estabiliza o acesso interno ao workload dentro do **Data Plane.**

O **Service** não substitui o gateway.

Ele prepara o caminho para que o tráfego roteado pelo gateway chegue corretamente ao runtime da aplicação.



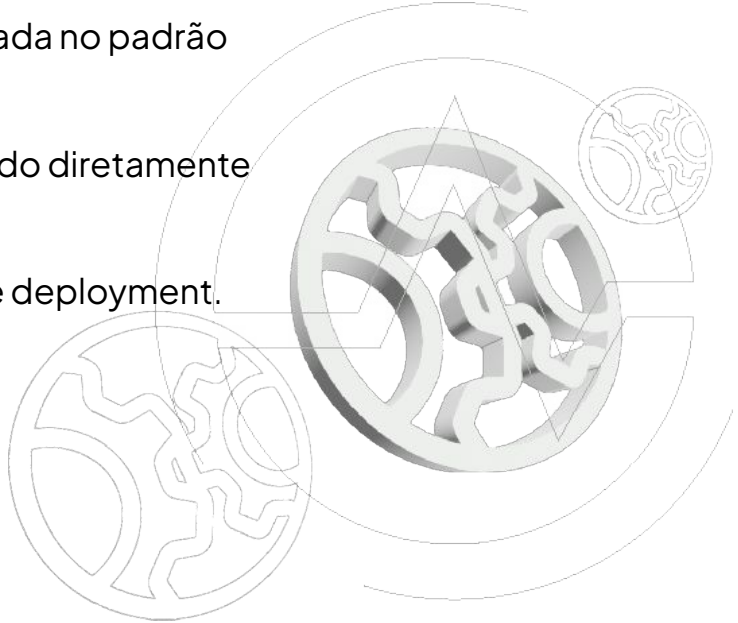
O papel do HTTPRoute

No **OpenChoreo**, o **HTTPRoute** é o recurso que configura o gateway para encaminhar tráfego externo até o **Service** da aplicação.

Isso significa que a exposição HTTP na plataforma é baseada no padrão Kubernetes Gateway API.

O **HTTPRoute** não é, no fluxo padrão, um artefato modelado diretamente pelo engenheiro.

Ele é gerado pela plataforma como parte do processo de deployment.



Gateway padrão do OpenChoreo

A documentação atual cita que o gateway padrão empacotado com o **OpenChoreo** é o kgateway.

Ele é habilitado por padrão ao instalar o **Data Plane**.

Ao mesmo tempo, a arquitetura modular permite usar outras implementações compatíveis com **Gateway API**.

Isso reforça que o **OpenChoreo** define contratos de plataforma, enquanto o módulo de gateway concretiza a implementação real.

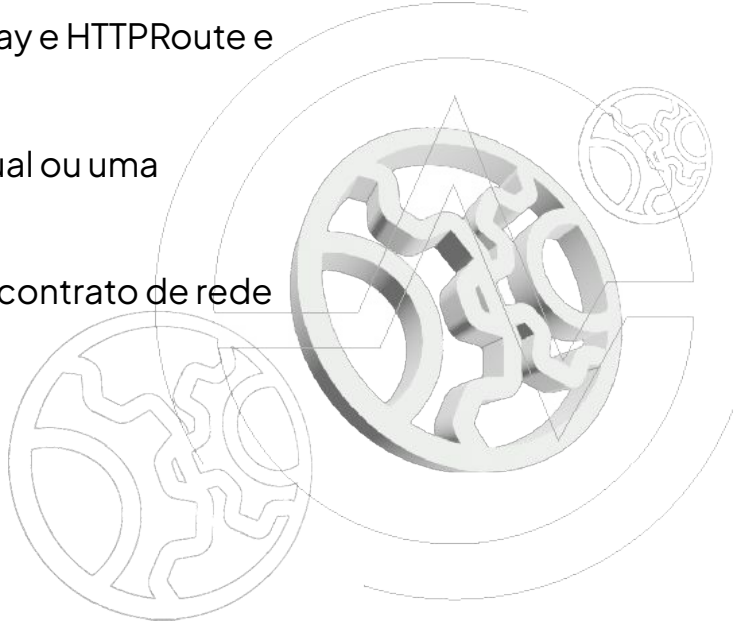
O gateway precisa obedecer ao contrato Gateway API

O módulo de **API Gateway** do **OpenChoreo** precisa fornecer uma implementação compatível com Kubernetes Gateway API.

O motivo é simples: o **Control Plane** cria recursos Gateway e HTTPRoute e espera que um controlador compatível os realize.

Portanto, o gateway no **OpenChoreo** não é um plugin visual ou uma abstração isolada.

Ele é um componente operacional que precisa cumprir o contrato de rede da plataforma.



Topologia de gateway: três dimensões

A topologia de gateway no **OpenChoreo** pode ser explicada por três dimensões:

- Direção do tráfego: ingress ou egress;
- Configuração de rede: external ou internal;
- Granularidade: compartilhado no Data Plane ou dedicado por Environment.

Essa estrutura evita explicações vagas como “tem um gateway na frente”.

Ela mostra que o gateway pode variar por direção, visibilidade de rede e nível de isolamento.

Topologia padrão: gateway externo compartilhado

Por padrão, o **OpenChoreo** instala um gateway de ingress externo no nível do **Data Plane**.

Com essa configuração, APIs e rotas implantadas em todos os environments ficam hospedadas como virtual hosts distintos no gateway compartilhado do **Data Plane**.

Essa topologia:

- Reduz custo de infraestrutura;
- Simplifica o ambiente;
- E é adequada para cenários básicos sem forte exigência de isolamento entre ambientes.

Este é o padrão mais comum para laboratórios introdutórios.

Gateway interno

Além do gateway externo, o **OpenChoreo** pode operar com gateway interno.

Esse modelo é útil quando o tráfego não deve ser exposto à internet pública.

A documentação atual aponta dois padrões comuns:

- ClusterIP, para acesso apenas dentro do cluster;
- LoadBalancer interno, para acesso dentro de VPC ou rede privada, sem exposição pública à internet.

A distinção entre external e internal é formal no modelo de configuração do **Data Plane**.

Gateway dedicado por Environment

O **OpenChoreo** também suporta gateway dedicado por **Environment**.

Nesse caso, o recurso **Environment** pode sobrescrever a configuração de gateway herdada do **Data Plane**.

Isso permite que um ambiente específico use:

- Host próprio;
- Porta própria;
- E topologia própria de exposição.

Esse modelo é útil quando um ambiente exige isolamento de tráfego, escala independente ou terminação TLS separada.

O que isso significa

Em um ambiente acadêmico, a topologia mais simples costuma ser:
gateway externo compartilhado no nível do **Data Plane**.

Esse modelo é suficiente para:

- Laboratórios introdutórios;
- Disciplinas com múltiplas turmas;
- E ambientes com baixo requisito de isolamento.

Em cenários mais avançados, a instituição pode adicionar:

- Gateways internos para acesso restrito;
- Ou gateways dedicados por **Environment** para maior isolamento.

A plataforma suporta essa evolução sem mudar o modelo mental ensinado aos estudantes.

Como responder à pergunta “onde meu serviço está exposto?”

A resposta correta é: Seu serviço é exposto no **Data Plane**.

O **endpoint** declarado no **Workload** informa à plataforma o que a aplicação oferece.

O **OpenChoreo** gera um **Service** e um **HTTPRoute** para esse **endpoint**.

O **HTTPRoute** é associado a um **Gateway** compatível com Kubernetes Gateway API, normalmente o gateway externo compartilhado do **Data Plane**, a menos que exista gateway interno ou override do **Environment**.

Essa é a resposta tecnicamente correta e pedagogicamente completa.

Erros conceituais a evitar

Erro 1: “O serviço fica exposto no Control Plane.”
Incorreto. Os recursos de runtime e exposição são aplicados no Data Plane.

Erro 2: “Endpoint é só uma porta.”
Incorreto. Endpoint é a declaração formal da interface de rede da aplicação.

Erro 3: “O aluno precisa criar manualmente HTTPRoute.”
Incorreto no fluxo padrão. O HTTPRoute é gerado pela plataforma.

Erro 4: “Qualquer gateway Kubernetes serve do mesmo jeito.”
Incorreto. O módulo precisa ser compatível com Kubernetes Gateway API.

Erro 5 “Cada ambiente já nasce com gateway dedicado.”
Incorreto. O padrão é gateway externo compartilhado no nível do Data Plane.

Síntese

No **OpenChoreo**, aplicações são expostas no **Data Plane**.

O **Workload** declara endpoints.

A plataforma gera **Service** e **HTTPRoute**.

Um gateway compatível com Kubernetes Gateway API, normalmente o gateway externo compartilhado do **Data Plane**, roteia o tráfego até o serviço correto.

O **Data Plane** pode também ter gateways internos, e o **Environment** pode sobrescrever o gateway padrão para cenários com isolamento por ambiente.

Esse modelo oferece ao ambiente acadêmico uma forma consistente, governada e evolutiva de publicar aplicações.

Módulo X: Autorização

Last Updated: April 2026

Autorização, tenancy e ambientes multiusuário seguros no OpenChoreo

Este módulo explica como o **OpenChoreo** controla quem pode fazer o quê, sobre quais recursos, em qual escopo e com qual efeito.

No **OpenChoreo**, autorização não é um detalhe administrativo secundário.

Ela é a base para operar a plataforma com:

- Isolamento entre equipes;
- Segurança em ambientes compartilhados;
- Governança por escopo;
- E autonomia controlada para estudantes, monitores, professores e administradores.

Em um ambiente acadêmico, este é o módulo que transforma a plataforma em um ambiente multiusuário seguro de verdade.

Mensagem central do módulo

No **OpenChoreo**, autorização não é uma camada acessória.

Ela é o mecanismo que define:

- Quem pode operar recursos da plataforma e da aplicação;
- Em que escopo;
- E com qual efeito.

Isso é o que permite autonomia sem perda de isolamento, governança ou segurança institucional.

O sistema de autorização embutido

O **OpenChoreo** inclui um sistema de autorização embutido, baseado em roles e bindings.

Esse sistema é configurável e gerido declarativamente por CRDs Kubernetes.

A documentação oficial também afirma que a autorização é habilitada por padrão.

Quando a autorização é desabilitada, todas as requisições passam a ser permitidas sem avaliação de política.

Essa configuração só deve ser usada em desenvolvimento ou teste.

Por que não desabilitar autorização

O padrão correto é manter a autorização habilitada.

Desabilitar autorização em um ambiente compartilhado elimina a principal barreira que impede:

- Acesso indevido entre equipes;
- Alteração acidental de recursos de outras turmas;
- Uso indevido de ambientes e pipelines institucionais;
- E perda de governança sobre contas de serviço e integrações.

A própria documentação alerta que desabilitar autorização remove todo o controle de acesso.

Os quatro elementos centrais do modelo

O modelo de autorização do **OpenChoreo** pode ser ensinado a partir de quatro elementos centrais:

Subject: quem está fazendo a requisição.

Role: quais ações são concedidas.

Scope: onde essas ações se aplicam.

Effect: se o binding concede ou nega o acesso.

A pergunta orientadora do modelo é: quem pode fazer o quê, e onde?

Subject

Subject é a identidade que faz uma requisição.

No **OpenChoreo**, essa identidade é reconhecida por entitlements, isto é, pares claim-valor extraídos do token JWT ou OIDC.

Exemplos típicos:

- Groups:platformEngineer
- sub:openchoreo-backstage-client
- Email:usuario@universidade.br

Um mesmo usuário pode ter múltiplos entitlements, e cada um deles pode ser avaliado pelo sistema de autorização.

Subject types e integração com o IdP institucional

Subject types fazem a ponte entre os claims do token da instituição e o mecanismo de autorização do **OpenChoreo**.

A documentação mostra, como padrão:

- User mapeando para o claim groups;
- Service Account mapeando para o claim sub.

Esse mapeamento é configurável.

Isso significa que o **OpenChoreo** não adivinha a estrutura do token da organização.

A equipe de plataforma precisa ajustar os subject types para refletir corretamente o provedor de identidade institucional.

Role

Role é uma coleção nomeada de ações permitidas.

As ações seguem o formato:
resource:verb

Exemplos:

- component:create
- project:view
- componenttype:create

Ações também aceitam curingas, como:

- component:*

Role não é, por si só, um perfil institucional como aluno, monitor ou professor.

Ela é apenas um conjunto de permissões operacionais.

Roles padrão

O **OpenChoreo** já fornece roles padrão que servem como ponto de partida.

A documentação atual destaca, entre outras:

- super-admin;
- backstage-catalog-reader;
- rca-agent.

A documentação também alerta que algumas roles e bindings internos são necessários para integrações da própria plataforma e não devem ser removidos sem pleno entendimento do impacto.

Isso é importante porque a autorização do **OpenChoreo** protege usuários e aplicações, mas também sustenta serviços internos da própria plataforma.

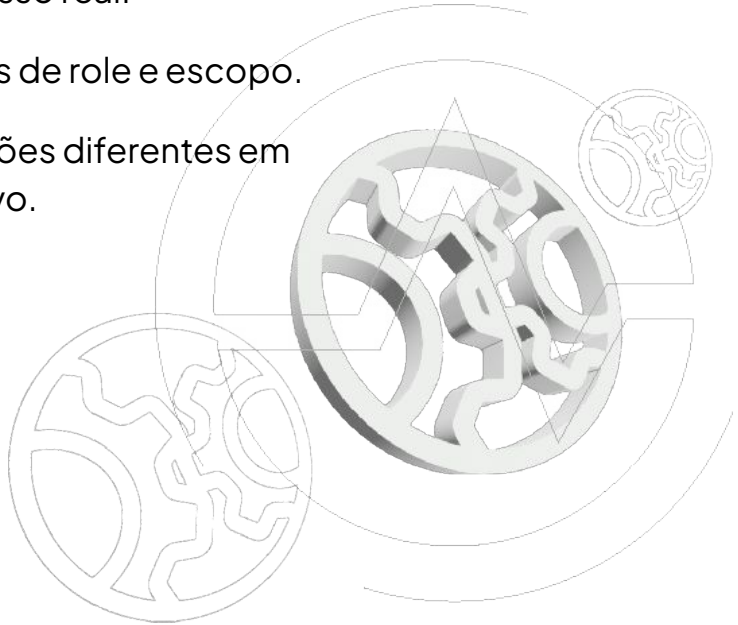
Role binding

Role binding é o recurso que conecta um **subject** a uma ou mais **roles**.

É o **binding** que transforma permissões abstratas em acesso real.

No **OpenChoreo**, um binding pode conter múltiplos pares de role e escopo.

Isso permite que uma mesma identidade receba permissões diferentes em escopos diferentes, dentro do mesmo modelo declarativo.



Effect: allow e deny

No **OpenChoreo**, bindings carregam um **effect**.

Esse effect pode ser:

- allow
- deny

A estratégia documentada é deny-overrides.

Isso significa:

- Se houver um binding allow correspondente e nenhum deny aplicável, a requisição é permitida;
- Se houver um deny aplicável, a requisição é negada;
- Se nenhum binding casar, a decisão padrão é negar.

A formulação correta é: deny sempre vence.

Hierarquia formal de recursos

A hierarquia formal de recursos no **OpenChoreo** é:

Cluster → Namespace → Project → Component

Essa hierarquia define a base do escopo das permissões.

Recursos cluster-scoped ficam no topo e não pertencem a **namespace** algum.

Permissões aplicadas em um nível mais alto podem alcançar níveis inferiores.

Permissões aplicadas em um nível mais baixo não sobem na hierarquia.

O que é scope

Scope é a fronteira que determina onde uma permissão realmente vale.

No **OpenChoreo**, o binding pode ser refinado até:

- Namespace;
- Project;
- E Component.

Omitir escopo significa aplicar a permissão no nível mais amplo permitido por aquele tipo de binding.

Esse é o mecanismo que permite modelar tenancy segura em universidade com granularidade real.

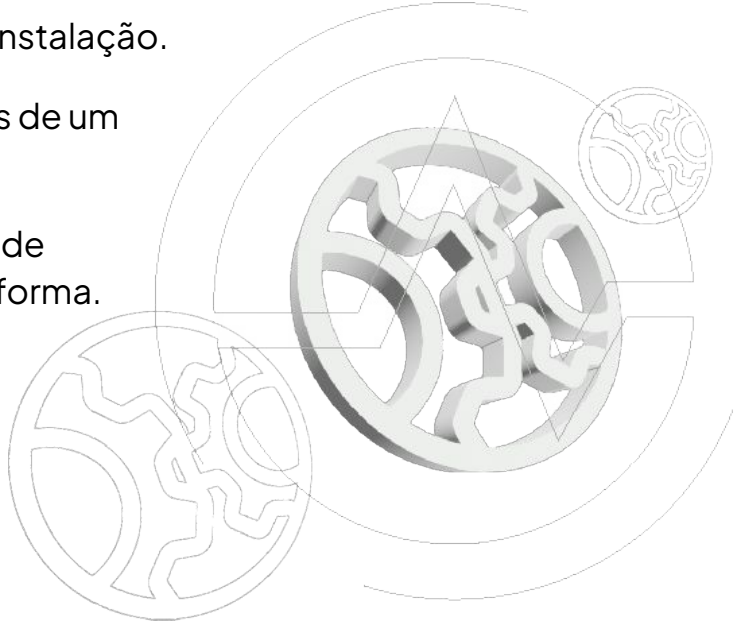
Cluster-scoped versus namespace-scoped

O **OpenChoreo** separa roles e bindings em dois grandes escopos administrativos:

Cluster-scoped: para permissões e padrões globais da instalação.

Namespace-scoped: para permissões e exceções locais de um namespace específico.

Essa distinção não é detalhe interno, ela define o modelo de compartilhamento e isolamento das permissões na plataforma.



Quando usar cluster-scoped e quando usar namespace-scoped

Use **cluster-scoped** quando a permissão precisar refletir um papel institucional global, como:

- Equipe central de plataforma;
- Administradores institucionais;
- Integrações internas da plataforma.

Use namespace-scoped quando a permissão precisar refletir um contexto local, como:

- Uma disciplina;
- Um laboratório;
- Uma turma;
- Ou uma unidade acadêmica específica.

Esse é o padrão correto para construir tenancy segura em ambiente acadêmico.

Access Control UI no Backstage

Na interface atual do **OpenChoreo**, a gestão operacional de autorização acontece em:

Settings → Access Control

A Access Control UI possui três seções:

- Roles;
- Role Bindings;
- Actions.

A seção **Roles** permite alternar entre Cluster Roles e Namespace Roles.

A seção **Role Bindings** permite conectar subjects a **roles** com escopo refinado.

A seção **Actions** permite navegar pelas ações disponíveis no sistema.

Criação de roles e bindings na UI

Na **Access Control UI**, a criação de roles pode usar templates rápidos como:

- Developer;
- Viewer;
- Admin.

Na criação de bindings, a interface usa um wizard orientado por etapas para:

- Selecionar o subject;
- Configurar os role mappings;
- Escolher o effect;
- E revisar a política antes de criar.

A própria UI permite afunilar o escopo até namespace, project e component. Isso torna a granularidade do modelo visível e operacional.

O que isso significa

Com a combinação de **roles**, **bindings**, **escopo** e **efeito**, a instituição pode modelar perfis como:

- Alunos com acesso apenas aos seus projetos;
- Monitores com leitura ampliada e alguns poderes operacionais;
- Professores com gestão sobre namespaces de disciplina;
- Administradores institucionais com permissões cluster-wide;
- Contas de serviço restritas a integrações específicas.

Esse é o mecanismo que impede interferência entre equipes e preserva governança em ambientes compartilhados.

Erros conceituais a evitar

Erro 1: “Se o usuário autenticou, já pode operar a plataforma.”
Incorreto. Autenticação e autorização são etapas diferentes.

Erro 2: “Role é a mesma coisa que perfil institucional.”
Incorreto. Role é um conjunto de ações; o perfil emerge com binding e escopo.

Erro 3: “Escopo só muda o que aparece na interface.”
Incorreto. Escopo muda a permissão efetiva.

Erro 4: “Permissões sempre se somam; deny não importa.”
Incorreto. Deny sempre vence quando aplicável.

Erro 5: “Desabilitar autorização é aceitável em ambiente de disciplina.”
Incorreto. A documentação recomenda isso apenas para desenvolvimento ou teste.

Erro 6: “Subject types são fixos.”
Incorreto. Eles são configuráveis e precisam refletir o IdP real da instalação.

Síntese

No **OpenChoreo**, autorização é um sistema RBAC declarativo e embutido, habilitado por padrão.

Ele combina:

- Subject;
- Role;
- Scope;
- E Effect

para decidir acesso a recursos da plataforma e das aplicações.

A hierarquia de escopo vai de **cluster** a **component**.

A separação entre **roles** e **bindings cluster-scoped** e **namespace-scoped** permite construir tenancy segura para ambientes universitários.

A **Access Control UI** do Backstage fornece a interface operacional para gerir essas políticas.

Módulo XI: Encerramento

Last Updated: April 2026

Encerramento: como ensinar estudantes a usar o OpenChoreo

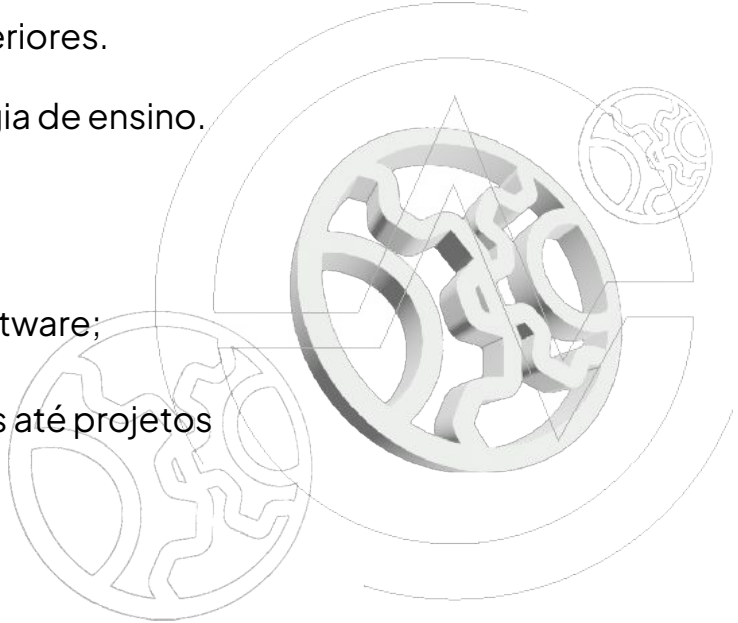
Este módulo encerra o treinamento conectando arquitetura, abstrações e operação do **OpenChoreo** ao contexto pedagógico acadêmico.

O objetivo não é repetir mecanicamente os módulos anteriores.

O objetivo é transformar o conteúdo técnico em estratégia de ensino.

O **OpenChoreo** deve ser apresentado como:

- Plataforma orientada por abstrações;
- Base para ensino progressivo de engenharia de software;
- Ambiente para governança acadêmica segura;
- E suporte para evolução de disciplinas introdutórias até projetos avançados.



Mensagem central do módulo

O **OpenChoreo** deve ser ensinado em ambiente academico como uma plataforma de engenharia de software orientada por abstrações.

Os estudantes devem aprender primeiro a modelar aplicações com:

- Projects;
- Components;
- Workloads.

A equipe docente e de plataforma deve governar:

- Namespaces;
- Environments;
- Pipelines;
- Component types;
- Traits;
- E Políticas.

À medida que o curso avança, observabilidade, promoção entre ambientes e integrações externas podem ser introduzidas como camadas adicionais de maturidade.

O OpenChoreo deve ser ensinado como plataforma, não como sequência de telas

A documentação oficial organiza o **OpenChoreo** em:

- Arquitetura;
- Conceitos;
- Guia do desenvolvedor;
- Guia do platform engineer;
- Runtime model;
- Resource relationships;
- E examples catalog.

Isso indica que o produto deve ser ensinado como modelo de plataforma, e não apenas como workflow de interface.

Ensinar somente a interface reduz o valor pedagógico do **OpenChoreo**.

Ensinar a plataforma como arquitetura, abstrações, lifecycle e operação produz um resultado formativo muito mais forte.

O que os estudantes devem aprender primeiro

O primeiro contato do estudante com o **OpenChoreo** deve acontecer pelas abstrações do desenvolvedor.

A documentação oficial define essas abstrações como um modelo declarativo para expressar:

- Arquitetura da aplicação;
- Dependências;
- Requisitos de runtime;
- E necessidades operacionais.

Por isso, o estudante deve começar por:

- Project;
- Component;
- Workload;
- Endpoints;
- Dependencies;
- Build a partir de fonte ou imagem pronta;
- E deploy básico.

Essa ordem respeita o desenho original da plataforma e reduz sobrecarga cognitiva.

O modelo mental inicial do estudante

Em termos pedagógicos, o estudante deve entender primeiro:

Project: o contexto principal da aplicação.

Component: a unidade implantável governada.

Workload: o contrato de runtime.

Endpoints: as interfaces expostas.

Dependencies: as relações explícitas entre componentes e serviços.

Build e deploy: os caminhos para transformar código ou imagem em sistema em execução.

Esse é o caminho mais correto para introdução ao OpenChoreo em disciplinas iniciais.

O mínimo viável de operação para o estudante

Em uma disciplina introdutória, o conteúdo mínimo recomendado ao estudante é:

- Consumir ou entrar em um **Project** já preparado;
- Criar um **Component**;
- Compreender ou definir o **Workload**;
- Usar imagem pronta ou build a partir de fonte;
- Acompanhar o deploy;
- Consultar logs e estados básicos de execução.

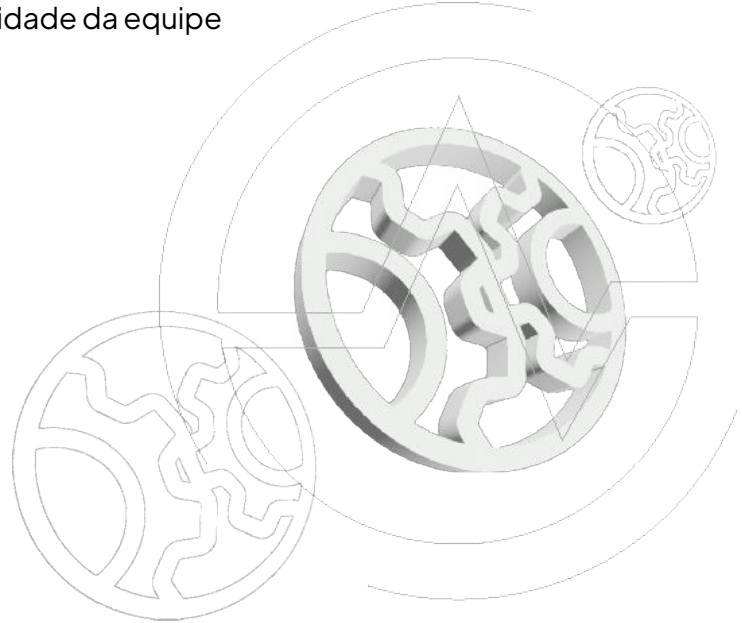
O estudante não precisa começar dominando a plataforma inteira. Ele precisa começar dominando o caminho declarativo de modelagem da aplicação.

O que deve permanecer com a equipe de plataforma

A documentação oficial define as abstrações de plataforma como a camada fundacional usada por platform engineers para construir e gerenciar a IDP.

Em ambiente acadêmico, isso deve permanecer como responsabilidade da equipe docente, dos TAs ou da equipe de plataforma:

- Namespaces;
- Environments;
- DataPlanes;
- WorkflowPlanes;
- DeploymentPipelines;
- ComponentTypes;
- Traits;
- Workflows;
- Autorização;
- Integração com identidade institucional;
- Observabilidade centralizada;
- E integrações externas.



Essa separação de responsabilidades é estrutural no **OpenChoreo**.

Por que essa separação é pedagogicamente correta

Separar o que pertence ao estudante do que pertence à equipe de plataforma evita dois problemas:

1. Sobrecarregar o aluno com governança e infraestrutura cedo demais;
2. Transformar o curso em uma sequência de cliques administrativos sem foco na modelagem da aplicação.

A própria documentação do **OpenChoreo** favorece um ensino em camadas:

- Desenvolvedor primeiro;
- Plataforma depois.

Essa separação melhora a experiência de aprendizagem e preserva a coerência do produto como plataforma.

Progressão sugerida para estudantes

Uma progressão pedagógica coerente para estudantes é:

1. Criação de **Project** e **Component**;
2. Definição de **Workload**;
3. Build a partir de fonte ou deploy de imagem pronta;
4. Promoção entre ambientes;
5. Observabilidade e debugging.

Essa ordem acompanha a própria organização da documentação oficial e favorece aprendizagem progressiva, com aumento gradual de maturidade operacional.

Etapa 1: criação de Project e Component

Na primeira etapa, o estudante deve aprender a modelar a aplicação em termos de domínio e unidades implantáveis.

Isso inclui:

- Entender **Project** como contexto principal;
- Entender **Component** como unidade implantável governada;
- E reconhecer que o projeto já nasce dentro de um contexto de entrega controlado.

Essa etapa deve enfatizar modelagem da aplicação, e não governança da plataforma.

Etapa 2: definição de Workload

Na segunda etapa, o estudante deve aprender o **Workload** como contrato de runtime.

Isso inclui:

- Imagem ou container;
- Endpoints;
- Dependencies;
- E requisitos imediatos de execução.

Esse é o momento ideal para consolidar a ideia de intenção declarativa: o estudante modela o que a aplicação precisa, e a plataforma materializa a infraestrutura correspondente.

Etapa 3: build a partir de fonte ou imagem pronta

A documentação oficial apresenta dois caminhos principais:

- Deploy a partir de imagem pronta;
- Build a partir de código-fonte.

Essa bifurcação é pedagogicamente útil.

Em disciplinas introdutórias, pode-se reduzir fricção usando imagem pronta ou CI interno simples.

Em disciplinas avançadas, pode-se explorar workflows, source build e integrações externas com mais profundidade.

O importante é que ambos os caminhos continuam dentro do mesmo modelo declarativo da plataforma.

Etapa 4: promoção entre ambientes

Depois que o estudante domina **Component** e **Workload**, a próxima camada de maturidade é a promoção entre ambientes.

Essa etapa permite ensinar:

- Diferença entre development, staging e production;
- Promoção governada;
- Reprodutibilidade;
- E separação entre configuração estável e variações ambientais.

Essa camada conecta o uso acadêmico do **OpenChoreo** a práticas modernas de engenharia de software.

Etapa 5: observabilidade e debugging

A etapa final da progressão estudantil deve incluir observabilidade e debugging.

Nesse ponto, o estudante passa de “subir a aplicação” para “observar, diagnosticar e operar a aplicação”.

Essa etapa é especialmente valiosa em:

- Projetos integradores;
- Capstones;
- TCCs;
- E disciplinas de arquitetura ou operação cloud-native.

Ela eleva o curso de deploy básico para operação consciente e diagnóstico estruturado.

Progressão sugerida para docentes e equipe acadêmica

Para docentes, TAs e mantenedores, a progressão recomendada é diferente.

A equipe acadêmica deve dominar, em ordem de maturidade:

1. **Namespaces**;
2. **Environments** e **DeploymentPipelines**;
3. **ComponentTypes** e **Traits**;
4. Autorização e integração com identidade;
5. Observabilidade, alerting e integração institucional;
6. E, quando necessário, extensibilidade da plataforma.

Essa é a camada de governança que sustenta o autosserviço seguro para estudantes.

O que o Examples Catalog demonstra

O **Examples Catalog** é particularmente importante para adoção acadêmica porque a própria documentação o organiza em duas grandes categorias:

- Platform Configuration;
- Application Deployment.

Isso confirma que o **OpenChoreo** é adequado tanto para:

- Ensinar configuração e governança da plataforma;
- Quanto para ensinar modelagem e implantação de aplicações.

Essa dupla vocação é uma das razões pelas quais o **OpenChoreo** se adapta bem ao ensino acadêmico.

Erros conceituais a evitar no onboarding

Erro 1: "Os estudantes devem começar configurando a plataforma."
Incorreto. O primeiro contato deve ser com abstrações do desenvolvedor.

Erro 2: "OpenChoreo serve apenas para deploy."
Incorreto. A documentação mostra uso tanto para configuração de plataforma quanto para implantação de aplicações.

Erro 3: "O melhor jeito de ensinar é fazer só uma demo da interface."
Incorreto. O produto foi documentado como plataforma com arquitetura, abstrações e relações de recursos.

Erro 4: "Todos os usuários precisam operar os mesmos recursos."
Incorreto. A plataforma separa papéis, escopos e camadas de responsabilidade.

Recomendação ao corpo docente

Ensinar **OpenChoreo** corretamente em ambiente acadêmico significa:

- Começar pela modelagem da aplicação;
- Manter a governança com a equipe de plataforma;
- Evoluir a maturidade do curso em camadas;
- Usar exemplos práticos para reduzir fricção;
- E tratar a plataforma como ambiente de engenharia, não apenas de deploy.

Essa abordagem produz melhor resultado pedagógico e melhor preparo profissional para os estudantes.

Thanks!

wso2.com